

Agentic Automation for SOAR with Zero Trust Policy Enforcement and RCF-Based Anomaly Scoring

A Report submitted
in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER ENGINEERING



Submitted By

Arib Ansari (22BCS009)

Ahal Khan (22BCS005)

Under the supervision of

Dr. Mumtaz Ahmed

(Associate Professor)

**DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY
JAMIA MILLIA ISLAMIA
NEW DELHI (110025)
(YEAR 2026)**

जामिया मिल्लिया इस्लामिया

(संसदीय अधिनियमानुसार केन्द्रीय विश्वविद्यालय)
मौलाना मोहम्मद अली जौहर मार्ग, नई दिल्ली-110025
JAMIA MILLIA ISLAMIA

NAAC Accredited Grade 'A++' (A Central University by an Act of Parliament)
Maulana Mohammed Ali Jauhar Marg, New Delhi-110025

दूरभाष

Tel. : 26980281, 26985831

विस्तार

Extn. : 2580

EPABX : 26981717

Extn. : 2442

E-mail : computerengg@jmi.ac.in

Web. : http://jmi.ac.in



कंप्यूटर अभियांत्रिकी विभाग
अभियांत्रिकी एवं प्रौद्योगिकी संकाय

Department of Computer Engineering
Faculty of Engineering and Technology

شعبہ کمپیوٹر انجینئرنگ
فیکلٹی آف انجینئرنگ اینڈ ٹیکنالوجی

Dated 15-05-2026.

CERTIFICATE

As per the declaration of the student this is to certify that the project report entitled **Agentic Automation for SOAR with Zero Trust Policy Enforcement and RCF-Based Anomaly Scoring** submitted by **Arib Ansari** (Roll No.: 22BCS009) and **Ahal Khan** (Roll No.: 22BCS005) to the Department of Computer Engineering, F/O Engineering & Technology, Jamia Millia Islamia New Delhi – 110025 in partial fulfillment for the award of the degree of B. Tech in (Computer Engineering) is a bona fide record of project work carried out by him/her under my/our supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

The aforementioned project has been pursued diligently and ethically by the students as a part of their Bachelor of Engineering curriculum in Computer Engineering at Jamia Millia Islamia, during the academic year 2025-26. This project is presented as a culmination of their academic pursuits and to fulfil the requisites for the successful completion of their degree program.

Prof. Mohd Amjad

(Head of the Department)

Department of Computer Engineering
Faculty of Engineering & Technology
JAMIA MILLIA ISLAMIA

Dr. Mumtaz Ahmed

(Associate Professor)

Department of Computer Engineering
Faculty of Engineering & Technology
JAMIA MILLIA ISLAMIA

DECLARATION

We, Arib Ansari (22BCS009), and Ahal Khan (22BCS005), solemnly declare that the project work entitled **Agentic Automation for SOAR with Zero Trust Policy Enforcement and RCF-Based Anomaly Scoring** is a record of our original work carried out under the supervision and guidance of **Dr. Mumtaz Ahmed**, Associate Professor, Department of Computer Engineering, Jamia Millia Islamia, New Delhi. The work presented in this project is authentic, and any external sources utilized have been duly acknowledged.

We declare that this project has not been submitted previously in part or in full for the award of any degree or diploma in this institute or any other university. All the data, figures, and concepts taken from other sources have been appropriately cited and referenced.

Arib Ansari
(22BCS009)

Ahal Khan
(22BCS005)

New Delhi-110025

May 2026

ACKNOWLEDGEMENT

We extend our heartfelt gratitude to **Dr. Mumtaz Ahmed**, Associate Professor, Department of Computer Engineering, Jamia Millia Islamia, New Delhi, for his unwavering support, expert guidance, and encouragement throughout the research endeavour. His profound knowledge, insightful inputs, and constant motivation were instrumental in shaping this project.

We would like to express our sincere appreciation to **Prof. Mohd Amjad**, Head of the Department of Computer Engineering, for his continuous support, invaluable advice, and belief in our capabilities. His encouragement and willingness to provide necessary resources have been pivotal in the successful completion of this project.

We are indebted to the entire faculty of the Department of Computer Engineering at Jamia Millia Islamia for imparting us with comprehensive knowledge and fostering an environment conducive to learning and innovation. Their teachings, guidance, and mentorship have been invaluable in shaping our academic journey.

Furthermore, we extend our heartfelt thanks to our classmates and friends for their support, valuable insights, and continuous encouragement throughout this endeavour. Their constructive feedback and discussions have been instrumental in refining our ideas and approach.

We would like to apologize for any unintentional omissions of names or contributions. The contributions of all involved are deeply appreciated and acknowledged.

Arib Ansari
(22BCS009)

Ahal Khan
(22BCS005)

ABSTRACT

Agentic Automation for Security Orchestration, Automation, and Response (SOAR) holds transformative potential at the intersection of autonomous artificial intelligence and cybersecurity, offering novel approaches to the limitations of static defense mechanisms. This project embarks on an exploration of the possibilities of **agentic SOAR** applications in the domain of **Zero Trust Architecture (ZTA)** and **probabilistic anomaly detection**. The fundamental idea is to leverage **autonomous agents** and **Random Cut Forest (RCF)** algorithms to enhance current solutions in analyzing and responding to evolving security threats.

The primary objectives encompass a thorough investigation of existing agentic frameworks and unsupervised machine learning techniques tailored for **continuous trust re-evaluation**. Implementation is executed using a multi-agent reasoning loop that integrates probabilistic anomaly scoring with dynamic policy enforcement. A comprehensive literature review contextualizes the current landscape of autonomous cyber defense, guiding the hands-on development of a **closed-loop feedback system**.

The anticipated outcomes of this project are two-fold:

1. **Demonstration of Agentic SOAR's Potential:** Illustrating the efficacy of autonomous agents in solving real-world security challenges, including the mitigation of alert fatigue through continuous behavioral monitoring and graduated enforcement actions such as step-up authentication or network isolation.
2. **Deeper Insight into Probabilistic Reasoning:** Gaining a nuanced understanding of the theoretical foundations and practical challenges of integrating RCF-based anomaly scoring and meta-learning within a Zero Trust environment.

This project contributes to the field of security software engineering through the development and empirical validation of an agentic, **risk-adaptive SOAR framework**. By leveraging the **UNSW-NB15 benchmark dataset**, the system's efficacy is evaluated using a multifaceted methodology. Quantitative performance is assessed through rigorous metrics while qualitative performance is measured by the framework's ability to autonomously synthesize **remediation rules** and maintain **self-healing operational stability**. A comparative analysis between traditional ML-based classification and the proposed agentic evaluation pipeline demonstrates the significant advantages of **decision-centric automation** in reducing alert fatigue.

TABLE OF CONTENTS

DESCRIPTION	Page No
CERTIFICATE	III
DECLARATION	V
ACKNOWLEDGEMENT	VII
ABSTRACT	IX
LIST OF FIGURES	XIX
LIST OF TABLES	XXIII
ABBREVIATIONS	XXV

Chapter 1: Introduction	1-20
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Overview of Security Automation	5
1.4.1 The evolution of SOAR	5
1.4.2 The challenge of alert fatigue	7
1.5 Principles of Zero Trust Architecture	9
1.5.1 Static vs continuous trust	9
1.5.2 Graduated enforcement	11
1.6 Machine Learning in Risk Analysis	11
1.6.1 The role of probabilistic scoring	11
1.6.2 Fusing categorical and anomaly signals	13
1.7 The Shift to Cognitive Cyber Defense	15
1.7.1 Limitations of deterministic playbooks	15
1.7.2 Agentic AI in SOCs	17
1.8 Primary Limitations	17
1.8.1 Scarcity of open source SOAR playbooks	17
1.8.2 Hardware constraints	19
1.8.3 Inference latency	19

1.8.4	Single-Event overfitting	19
Chapter 2:	Literature Review	21-42
2.1	Security Orchestration, Automation and response	21
2.1.1	Introduction	21
2.1.2	Current frameworks and limitations	21
2.2	Adoption of Zero Trust Models	23
2.3	Machine Learning in Anomaly Detection	27
2.3.1	Categorical threat classification	27
2.3.2	Probabilistic anomaly detection	29
2.3.3	Meta learning for score fusion	31
2.4	Agentic AI in Cyber Defense	33
2.4.1	Static automation to autonomous workflows	33
2.4.2	Evaluating agentic security constraints	35
2.5	Research gap	37
Chapter 3:	Proposed Methodology	43-84
3.1	System architecture overview	43
3.2	Dataset and preprocessing	49
3.2.1	The UNSW-NB15 dataset	49
3.2.2	Feature engineering	53
3.2.3	Preventing data leakage: OOF Training	55
3.3	The ML Detection Layer	55
3.3.1	Binary threat classification with CatBoost	55
3.3.2	Multi-class incident agent and probability calibration	57
3.3.3	Anomaly scoring with robust random cut forest	59
3.4	The Cost-Sensitive Meta-Learner (Fusion Layer)	61
3.4.1	Meta-Feature construction	61
3.4.2	Cost-Sensitive training and threshold optimisation	65
3.5	The Zero Trust Policy Agent	67
3.5.1	Trust score computation	67
3.5.2	Behaviour signatures and the evidence ledger	71
3.6	The LLM-Driven Response Agent	75
3.6.1	Prompt engineering	75
3.6.2	SOAR playbook evolution	77

3.7 Autonomous Self Healing Loop	79
3.7.1 Vector Memory Architecture	79
3.7.2 False positive correction	79
3.7.3 False negative correction	81
3.7.4 Rule insertion and deduplication	83
3.7.5 Periodic consolidation	83
Chapter 4: Results and Analysis	85-124
4.1 Experimental setup and evaluation methodology	85
4.1.1 Evaluation Metrics	87
4.2 ML Layer Performance	93
4.2.1 CatBoost and incident agent classification results	93
4.2.2 RCF score distribution and calibration analysis	95
4.3 Meta-Learner and Fusion Performance	99
4.4 End-to-End Pipeline Evaluation	103
4.4.1 Evaluation setup and constraints	103
4.4.2 Classification performance and SOC impact	105
4.4.3 Unsupervised saturation and distribution shift analysis	107
4.4.4 Performance comparison	109
4.5 AI Agent Evaluation	111
4.5.1 SOAR pipeline analysis	111
4.5.2 Performance metrics	117
4.6 Self-Learning Loop	119
4.6.1 Correction activity and rule synthesis	119
4.6.2 Performance metrics	123
Chapter 5: Conclusion and Future Scope	125-132
5.1 Limitations	125
5.1.1 Latency Bottlenecks and "Fail-Safe" Degradation	125
5.1.2 VRAM Contention	127
5.1.3 Vulnerability to Adversarial Prompting	127
5.2 Future Directives	129
5.2.1 Dynamic RCF Re-calibration for Concept Drift	129
5.2.2 Interpretability vs. Justified Hallucination	129

5.2.3 Human-in-the-Loop (HITL) Verification	131
5.2.4 Cloud-Native SIEM Integration	131
Chapter 6: Conclusion	133
Chapter 7: References	135

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
1.1.	Agentic SOAR	1
1.2.	SOAR capabilities	7
1.3.	Zero Trust Architecture	9
2.1.	General meta-learning model based on stacked ensemble	33
2.2.	AgentSOC Architecture	35
3.1.	ML Training Pipeline	43
3.2.	Agentic SOAR Pipeline	45
3.3.	Self Learning Pipeline	47
3.4.	Analysis Prompt	81
4.1.	CatBoost and Incident Standalone Metrics	95
4.2.	RCF Standalone matrices	97
4.3.	ROC Curve Comparison Between CatBoost and RCF	99
4.4.	Optimal Weights and Thresholds	101
4.5.	Confusion Matrix - Pipeline Metrics	105
4.6.	Fusion Model Test Evaluation	107
4.7.	Fast-discard for a confirmed benign event	111
4.8.	Fast-Path for events within Trust Score	113
4.9.	LLM Escalation	113

FIGURE	TITLE	PAGE NO.
4.10.	LLM Skip via Policy Quarantine	115
4.11	Evidence ledger state after 100 events	115
4.12.	Self-Learning Loop in Action	123
5.1.	LLM Response Times	125

LIST OF TABLES

TABLE	TITLE	PAGE NO.
3.1.	Features of UNSW-NB15 dataset	51
4.1.	Quantitative Performance Comparison of Detection Components and Final Pipeline	109
4.2.	Agentic Routing Breakdown	117
4.3.	Synthesized Patch Rules	121

ABBREVIATIONS/ NOTATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
DDoS	Distributed Denial of Service
DNS	Domain Name System
GPU	Graphics Processing Unit
LLM	Large Language Model
ML	Machine Learning
MTTD	Mean Time To Detect
MTTR	Mean Time To Respond
OOF	Out of Fold
PCA	Principal Component Analysis
RCF	Random Cut Forest
ROC-AUC	Receiver Operating Characteristic-Area Under Curve
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
VRAM	Video Random Access Memory
ZTA	Zero Trust Architecture

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Enterprise security has fundamentally transitioned from a static, perimeter-based defense model to a landscape defined by continuous behavioural observation. As applications expand across hybrid cloud platforms and unmanaged remote endpoints, the volume of telemetry generated by identity systems and network logs has increased exponentially, often overwhelming Security Operations Centers (SOCs). While Security Orchestration, Automation, and Response (SOAR) platforms were introduced to mitigate this operational burden, their current implementations are largely restricted by manual configuration and static logic.

Legacy SOCs rely on rigid, rule-based systems that map alerts to binary "allow or block" responses. These systems frequently struggle to process ambiguous or evolving behavioral data, leading to delayed responses, inconsistent decision-making, and significant alert fatigue for security analysts. Furthermore, the rise of Zero Trust Architecture (ZTA) necessitates a shift toward continuous trust re-evaluation, a requirement that static execution pipelines cannot satisfy.

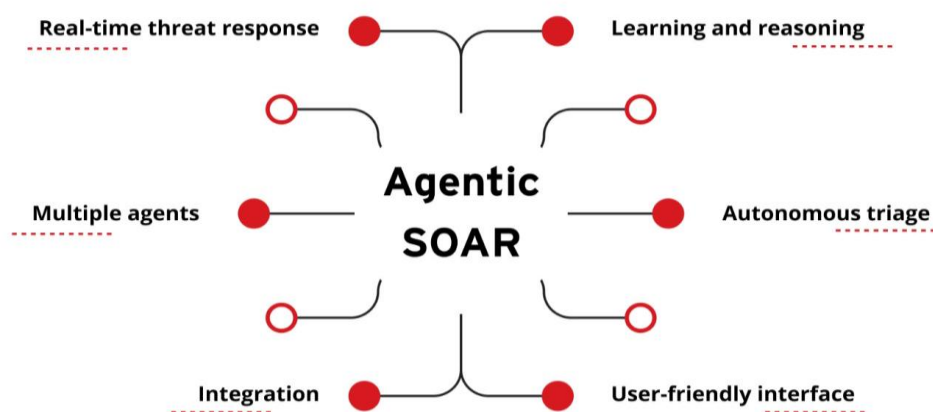


Fig 1.1 Agentic SOAR

There is a critical need for a decision-centric automation framework that treats security as a dynamic process rather than a fixed pipeline. By integrating Agentic Artificial Intelligence (AI)

with unsupervised anomaly detection, specifically the Robust Random Cut Forest (RCF) algorithm, security systems can generate probabilistic confidence signals to guide adaptive responses. This approach provides the necessary foundation for aligning automated response workflows with the core principles of Zero Trust.

1.2 Motivation

Despite the advancement of automated security tools, current SOAR frameworks exhibit several systemic limitations that undermine the efficacy of modern SOC operations:

Deterministic and Binary Decision-Making: Most automation platforms operate on deterministic logic, mapping alerts to playbooks with an assumed certainty that rarely exists in real-world incidents. This leads to a binary "allow or block" stance that lacks the nuance required to handle emerging or subtle behavioral threats.

Inability to Reason Under Uncertainty: Unsupervised techniques like RCF produce continuous anomaly scores, yet existing SOAR platforms lack the mechanisms to incorporate these probabilistic outputs into response logic.

Static Zero Trust Enforcement: While Zero Trust mandates that trust be continuously recalculated, current frameworks provide limited support for translating evolving risk signals into graduated enforcement actions.

Alert Fatigue and Distribution Shifts: Existing systems often fail to account for the severe class imbalances and distribution shifts native to enterprise telemetry, obscuring genuine security breaches.

This research investigates the development of an Agentic SOAR framework capable of reasoning under uncertainty. By utilizing a multi-agent reasoning loop and ensemble intelligence (CatBoost and RCF), the proposed system seeks to bridge the gap between probabilistic risk scoring and adaptive, Zero Trust-aligned enforcement.

1.3 Objectives

The primary objective of this research is to explore the potential of Agentic Artificial Intelligence and probabilistic machine learning in advancing SOAR systems. The specific goals are as follows:

Implement Ensemble Threat Detection via Meta-Learning: To develop a robust detection mechanism that utilizes a cost-sensitive meta-learner to fuse categorical threat probabilities from a CatBoost model with continuous, probabilistic anomaly scores from Random Cut Forest (RCF) algorithm. This ensemble approach enables the generation of holistic risk scores that effectively isolate both known threats and novel behavioral anomalies from background network noise.

Enable Continuous Trust Re-evaluation: To design a dynamic policy enforcement layer capable of continuously recalculating access decisions based on live telemetry, ensuring strict alignment with ZTA principles.

Develop a Multi-Agent Reasoning Loop: To architect a collaborative framework—incorporating Risk, Policy, and Response agents that translate evolving security signals into graduated enforcement actions, moving beyond the rigid, binary logic of traditional playbooks.

Engineer Autonomous Self-Healing Capabilities: To integrate a cognitive feedback mechanism that conducts root-cause analysis on operational misclassifications (false positives and false negatives) and autonomously synthesizes new routing rules to systematically reduce alert fatigue.

Empirical Validation and Benchmarking: To rigorously evaluate the proposed framework's efficacy and operational cost reduction using the UNSW-NB15 intrusion detection dataset, demonstrating its resilience against severe class imbalances and distribution shifts.

1.4 Overview of Security Automation

In modern enterprise environments, the sheer volume, velocity, and complexity of network telemetry, authentication logs, and endpoint data have rendered traditional, manual threat hunting practically unsustainable. By leveraging integrated APIs, algorithmic detection models, and rule-based execution paths, automation platforms continuously monitor digital ecosystems. This mechanized approach allows security systems to rapidly identify, triage, and remediate malicious activities at machine speed, thereby allowing organizations to transition from reactive alert management to proactive, strategic threat mitigation.

1.4.1 The evolution of SOAR

Historically, organizations relied on Security Information and Event Management (SIEM) platforms to aggregate logs and provide centralized visibility. However, as the velocity of cyber

threats escalated, the manual triaging of SIEM alerts became a severe operational bottleneck. To address this, **Security Orchestration, Automation, and Response (SOAR)** platforms were introduced to manage the full lifecycle of an incident. As defined by industry standards, a comprehensive SOAR platform operates across four primary capabilities:



Figure 1.2: SOAR capabilities

By automating these routine tasks, SOAR aims to significantly reduce the Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR), freeing security analysts to focus on complex, high-priority threats.

1.4.2 The challenge of alert fatigue

Despite the operational efficiencies introduced by SOAR, these platforms suffer from a critical systemic limitation: vulnerability to alert fatigue. Real-world Security Operations Centers operate in environments characterized by severe class imbalances, where benign network anomalies vastly outnumber genuine malicious intrusions.

Because legacy systems cannot adapt to modern distribution shifts or novel threat signatures, they trigger thousands of unjustified alerts. This overwhelming volume creates a "needle in a haystack" scenario, leading to severe alert fatigue. It not only degrades the operational

efficiency of the security team but also significantly increases the risk that a genuine, critical breach(false negative) will go unnoticed amidst the operational noise.

1.5 Principles of Zero Trust Architecture

1.5.1 Static vs. continuous trust evaluation

Zero Trust Architecture (ZTA) fundamentally alters the cybersecurity paradigm by eliminating the concept of a trusted internal network perimeter. The core tenet of ZTA is that trust must never be implicitly assumed based on a user's location, device, or prior authentication status. However, many early and traditional Zero Trust implementations enforce static policies, acting as a one-time gateway; once a session is granted, the system fails to adapt to shifting behavioral risks during that session.

To achieve true Zero Trust, access decisions must transition from static checkpoints to **continuous trust re-evaluation**. Within the context of this project, this principle is operationalized through a dedicated Policy Agent. Rather than making binary access decisions, the system continuously ingests live telemetry such as connection volumes, time-of-day access patterns, and historical quarantine data to compute a dynamic **Trust Score**. This score acts as a living metric that decays or recovers based on real-time network behavior, ensuring that the security posture of the environment evolves continuously alongside the active session.

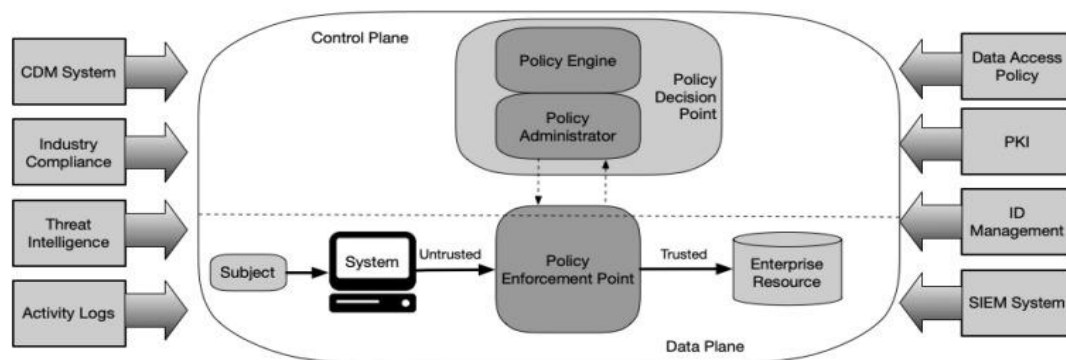


Figure 1.3: NIST Zero Trust Architecture [13]

1.5.2 Graduated enforcement and adaptive response

A direct consequence of continuous trust evaluation is the necessity for dynamic, proportional response mechanisms. Traditional security automation relies heavily on deterministic "allow

or block" paradigms, which struggle to process the probabilistic uncertainty of modern, evolving threats. When a behavioral anomaly emerges gradually and carries only partial confidence, executing an immediate, total network block often results in severe business disruption and alert fatigue.

To align strictly with Zero Trust principles, the proposed framework implements **graduated enforcement**, ensuring that the intensity of the automated response scales proportionally with the calculated risk. Instead of mapping alerts to rigid, static playbooks, the architecture leverages an LLM-driven Response Agent to evaluate the dynamic Trust Score against specific threat classifications and organizational constraints. This enables a context-aware spectrum of adaptive actions.

Example: a low-confidence anomaly might trigger proportional rate-limiting, while mathematically verified benign traffic is autonomously routed to a dynamic allowlist to reduce future computational overhead. Conversely, high-severity threshold violations trigger immediate and decisive containment, such as targeted network isolation.

By decoupling threat detection from fixed execution pipelines, the system ensures that mitigation strategies are both operationally resilient and contextually precise.

1.6 Machine Learning in Risk Analysis

1.6.1 The role of probabilistic scoring in threat detection

Traditional threat detection mechanisms typically rely on deterministic binary classification, mapping network events strictly to "benign" or "malicious" labels. This rigid categorization struggles to handle the probabilistic uncertainty inherent in zero-day exploits and subtle behavioral shifts. To overcome this limitation, machine learning transforms risk analysis into a continuous, probabilistic science. By utilizing unsupervised anomaly detection algorithms i.e the Robust Random Cut Forest (RCF), the system establishes a mathematically pure behavioral baseline by training exclusively on normal network traffic.

Rather than outputting a binary alert, the RCF algorithm quantifies the degree of deviation from this established baseline, producing a continuous anomaly score. To ensure this score maintains its dynamic range and does not prematurely saturate when evaluating severe anomalies, a linear percentile stretch calibration is applied, anchoring the scores between specific baseline

percentiles. This continuous probabilistic scoring allows the system to reason about the relative uncertainty of a threat, providing the necessary gradient of confidence required to trigger adaptive, proportional security responses rather than defaulting to immediate network isolation.

1.6.2 Fusing categorical and anomaly-based risk signals

Relying on a single detection methodology often leaves a security framework vulnerable to blind spots; anomaly detection may flag benign operational shifts as threats, while signature-based classifiers may miss entirely novel attacks. To achieve a holistic understanding of network incidents, this framework implements an ensemble intelligence architecture that fuses multiple distinct risk signals into a single, unified metric.

This fusion is orchestrated by a cost-sensitive meta-learner (L2 regularized logistic regression) model which ingests three primary meta-features:

1. **Categorical Risk:** The explicit probability of a known threat signature, generated by a highly regularized CatBoost binary classifier.
2. **Anomaly Risk:** The continuous deviation score generated by the RCF algorithm.
3. **Incident Uncertainty (Entropy):** A calculated metric derived from the Shannon entropy of a multi-class threat classifier's output distribution.

The inclusion of Shannon entropy is a critical engineering feature. When the multi-class agent is highly uncertain about an event, the resulting high entropy serves as a powerful signal that the traffic represents a borderline or novel threat requiring heightened scrutiny. By fusing these elements with a cost-sensitive function that heavily penalizes False Negatives (missed attacks) over False Positives (unjustified alerts), the meta-learner calculates a highly optimized **fused_risk** score. This final risk score acts as the definitive intelligence gateway, cleanly translating complex, multi-dimensional machine learning outputs into the dynamic trust evaluation pipeline.

1.7 The Shift to Cognitive Cyber Defense

As enterprise architectures evolve toward continuous trust evaluation, the mechanisms used to defend them must undergo a fundamental paradigm shift. Traditional automation is no longer sufficient; defensive systems must become cognitive.

1.7.1 Limitations of deterministic playbooks

Traditional Security Orchestration, Automation, and Response (SOAR) platforms operate on a fundamentally rigid premise: incident response can be codified into predefined "if/then" execution pipelines. While this deterministic logic excels at mitigating known, highly predictable threat signatures, it completely breaks down when confronted with the probabilistic uncertainty of modern cyberattacks.

When complex, multi-stage behavioural deviations are forced through static decision trees, critical contextual nuance is lost. This reliance on fixed thresholds creates several critical operational bottlenecks:

Binary Enforcement Constraints: Deterministic platforms are restricted to a binary "allow or block" stance, stripping away the ability to apply graduated, proportionate responses to ambiguously developing threats.

Incompatibility with Probabilistic Scoring: Static playbooks cannot ingest or interpret the continuous confidence scores generated by unsupervised anomaly detection models (such as Random Cut Forests) without reducing them to hard, context-blind thresholds.

Amplification of Alert Fatigue: Because they cannot adapt to distribution shifts or evolving baseline telemetry, static rules frequently trigger thousands of unjustified alerts (false positives), overwhelming human analysts and obscuring genuine breaches.

However, the ideal architecture provides a "best-of-both-worlds" scenario. In this advanced setup, deterministic if-then logic is not discarded; instead, it acts as strict, safe guardrails that allow probabilistic AI agents to reason and operate independently without taking destructive actions. By framing deterministic playbooks as policy constraints rather than exhaustive execution paths, the framework prevents catastrophic automated actions (such as isolating critical infrastructure due to a false positive) while still enabling the cognitive flexibility required to investigate ambiguous threats.

1.7.2 Agentic artificial intelligence in the SOC

Rather than relying on hardcoded execution paths, an Agentic SOAR framework utilizes autonomous reasoning agents that treat incident response as a dynamic, context-aware decision process.

By replacing static playbooks with a collaborative multi-agent reasoning loop, the system is capable of simulating human-like deduction at machine speed. In the proposed architecture, a deterministic Zero Trust Policy Agent continuously computes trust scores against organizational constraints, acting as a strict operational guardrail. When trust is breached, a Large Language Model (LLM)-driven Response Agent autonomously evaluates the probabilistic risk alongside the specific network telemetry to orchestrate a proportionate, context-aware response.

Crucially, this agentic ecosystem introduces the capability for "autonomous self-healing." A dedicated Playbook Editor Agent actively monitors the system for operational misclassifications, performs root-cause analysis on false positives and negatives, and dynamically synthesizes new routing rules. This allows the defensive posture of the SOC to continuously learn and evolve without requiring constant manual playbook re-engineering.

1.8 Primary Limitations

While the proposed Agentic SOAR framework presents a robust theoretical and practical advancement over traditional static security automation, its development and operational deployment are subject to several inherent limitations.

1.8.1 Scarcity of open-source SOAR playbooks

A significant hurdle in developing realistic security automation is the lack of standardized, enterprise-grade open-source SOAR playbooks. Commercial SOAR platforms (such as Palo Alto Cortex XSOAR or Splunk SOAR) rely on highly proprietary, closed-ecosystem playbooks that are rarely shared publicly. Consequently, the automated response workflows utilized in this research (e.g. NETWORK_ISOLATION, STEP_UP_AUTH) had to be synthesized as proxy representations. While these custom playbooks are functionally complete for experimental validation, they may not capture the full integration complexity of a multi-vendor enterprise SOC environment.

1.8.2 Hardware and computational constraints

The transition from deterministic logic to Cognitive Cyber Defense introduces substantial computational overhead. The architecture demands the concurrent execution of ensemble machine learning models (CatBoost and RCF) alongside a locally hosted Large Language

Model (LLM, specifically **deepseek-r1:8b**). Operating this multi-agent ecosystem locally on a consumer-grade GPU (such as an RTX 4060 with 8GB of VRAM) creates severe resource bottlenecks. Low VRAM capacity restricts the context window size the LLM can process at once and limits the batch processing capabilities of the anomaly detection layer, forcing sequential rather than highly parallelized event evaluation.

1.8.3 Inference latency

Directly tied to the hardware constraints is the challenge of inference latency. Traditional static playbooks execute in milliseconds, allowing them to keep pace with high-velocity network traffic. In contrast, invoking an LLM-driven Response Agent to read telemetry, evaluate trust score violations, and synthesize a reasoned JSON response takes several seconds per event. If the LLM experiences a timeout (e.g., exceeding a 240-second threshold), the system is forced to rely on a hardcoded fail-safe mechanism, temporarily nullifying the advantages of agentic reasoning.

1.8.4 Single-event overfitting

The PlaybookEditorAgent introduces the ability to autonomously write new routing rules based on false positive or false negative audits. A core challenge of this self-healing mechanism is the risk of "single-event overfitting." When the AI writes a patch rule to correct a mistake, it tends to anchor the rule on hyper-specific telemetry values (such as exact source and destination packet counts). Without aggressive, periodic rule consolidation, the system risks bloating the playbook with thousands of non-generalizable rules, which can degrade future routing efficiency and create conflicting policy enforcement.

CHAPTER 2

LITERATURE REVIEW

2.1 Security Orchestration, Automation and Response

2.1.1 Introduction

The conceptualization of Security Orchestration, Automation, and Response (SOAR) emerged as a necessary operational bridge between threat detection and threat containment. As enterprise networks expanded, the volume of telemetry rapidly outpaced human cognitive capacity, necessitating platforms that could automate portions of the incident response lifecycle and reduce the operational burden on security analysts.

2.1.2 Current frameworks and limitations

Current industry standards define SOAR as a technology that coordinates disparate security tools, automates routine tasks via deterministic playbooks, and standardizes incident response procedures. While the literature widely acknowledges that SOAR platforms successfully reduce the Mean Time to Respond (MTTR) for known, signature-based attacks, recent studies highlight severe architectural limitations when these systems encounter probabilistic uncertainty.

The Rigidity of Deterministic Playbooks: Traditional SOAR deployments are fundamentally rigid; they function effectively only when an incident closely follows expected, hardcoded scenarios. Ismail et al. (2025) observe that without a hyper-automation approach, SOC's relying on traditional SOAR are ill-equipped to handle the nuance of modern threats. This deterministic limitation becomes a critical failure point because static playbooks map alerts to binary "allow or block" responses, stripping away the continuous risk signals required to effectively triage ambiguous events.

Vulnerability to High-Volume and AI-Driven Attacks: This rigidity is further exposed when systems face complex, high-velocity intrusions. Alliouche and Chenni (2025), in a comprehensive study evaluating SOAR technologies for AI-based DDoS detection, emphasize that traditional, static rule-sets are easily overwhelmed by the rapid distribution shifts characteristic of automated attacks. Their research indicates that without integrating dynamic,

AI-driven reasoning directly into the response pipeline, traditional SOAR frameworks inevitably succumb to alert fatigue and systemic delays.

The Operational Gap: The consensus across contemporary literature (Oesch et al., 2024; Roy and Singh, 2026) is that the primary operational bottleneck in modern SOC is driven by the inability of static tools to adapt. For SOAR platforms to remain viable against sophisticated intrusions, the underlying architecture must transition from a fixed execution pipeline to a risk-adaptive, intelligent decision engine.

2.2 Adoption of Zero Trust Models

The rapid evolution of enterprise computing environments has fundamentally weakened the assumptions underlying traditional perimeter-based cybersecurity models. Conventional security architectures operate on the belief that entities inside an organizational network can be implicitly trusted once authenticated. However, the widespread adoption of cloud infrastructure, remote work, mobile devices, Internet of Things (IoT) ecosystems, and hybrid enterprise environments has dissolved the concept of a clearly defined network perimeter. As a result, modern attack campaigns increasingly exploit lateral movement, credential compromise, insider threats, and distributed access channels that perimeter-centric defenses cannot effectively contain.

To address these limitations, Zero Trust Architecture (ZTA) emerged as a security paradigm centered on the principle of “never trust, always verify.” Rose et al. (2020), in the foundational NIST Special Publication 800-207, formally standardized Zero Trust principles by asserting that no user, device, workload, or network flow should be inherently trusted based solely on location or prior authentication state. Instead, access decisions must be continuously validated through strict identity verification, least-privilege enforcement, contextual policy evaluation, and continuous monitoring of behavioral telemetry. Similarly, Syed et al. (2022), in a comprehensive survey of Zero Trust frameworks, emphasize that ZTA fundamentally shifts cybersecurity from static perimeter defense toward identity-centric and context-aware security models capable of operating within highly distributed enterprise environments.

The operationalization of Zero Trust was further accelerated through industry implementations such as Google’s BeyondCorp architecture. Ward and Beyer (2014) proposed BeyondCorp as a practical enterprise security model that eliminates implicit network trust by evaluating device

posture, user identity, and contextual signals before granting access to organizational resources. Rather than relying on VPN-based perimeter defenses, BeyondCorp treats every access request as originating from an untrusted environment, thereby enabling secure access control across distributed networks and remote infrastructures. This approach demonstrated that continuous verification could be implemented at enterprise scale and significantly influenced subsequent Zero Trust research and industrial adoption.

A defining characteristic of modern Zero Trust systems is the transition from static authentication toward continuous trust evaluation. Unlike traditional security models where trust is established once during login, ZTA requires trust levels to evolve dynamically throughout the lifetime of a session. Ojo (2025), in an evaluation of Zero Trust adoption for critical infrastructure protection, highlights that static access policies are insufficient in environments characterized by rapidly changing operational contexts and sophisticated attack surfaces. If post-authentication telemetry exhibits anomalous behavior such as unusual connection patterns, privilege escalation attempts, or suspicious lateral movement, the system must continuously re-evaluate and potentially revoke access privileges in real time.

Recent literature increasingly proposes integrating intelligent and adaptive mechanisms into Zero Trust ecosystems to address this requirement for continuous trust recalculation. Ramezanzpour and Jagannath (2021) argue that next-generation Zero Trust systems for 5G and 6G environments must incorporate machine learning-driven behavioral analysis to support intelligent trust evaluation under highly dynamic network conditions. Their work emphasizes that static rule-based authorization mechanisms are incapable of processing the scale, velocity, and uncertainty associated with modern distributed infrastructures. Similarly, Chen et al. (2022) identify that future Zero Trust implementations must evolve toward adaptive architectures capable of responding autonomously to evolving threats within decentralized cloud-native ecosystems.

Another major area of research focuses on trust-score computation and adaptive risk evaluation. Park et al. (2023) propose a dynamic endpoint risk-scoring framework in which device behavior, contextual telemetry, and historical activity are continuously evaluated to calculate evolving trust levels. Their research demonstrates that Zero Trust enforcement becomes significantly more effective when access decisions are based on probabilistic behavioral risk rather than static policy conditions alone. This concept directly aligns with

contemporary cybersecurity trends that treat trust not as a binary state, but as a continuously changing probabilistic metric influenced by real-time environmental conditions.

Despite these advancements, the literature reveals a critical operational limitation in existing Zero Trust implementations. Most current deployments still rely heavily on deterministic thresholds and predefined enforcement logic that lack contextual reasoning capabilities. While they can detect policy violations, they often cannot determine the severity, uncertainty, or behavioral implications of anomalous activity in a nuanced manner. Consequently, existing systems struggle to translate probabilistic behavioral intelligence into adaptive and proportionate response actions.

Collectively, the literature indicates that Zero Trust is evolving from a static identity-verification framework into a continuously adaptive risk-evaluation architecture. However, there remains a distinct lack of integrated systems capable of combining probabilistic anomaly detection, machine learning-driven trust evaluation, and autonomous policy orchestration into real-time SOC operations. This limitation establishes the foundation for integrating intelligent anomaly scoring and agentic decision-making mechanisms within Zero Trust-oriented cyber defense frameworks.

2.3 Machine Learning in Anomaly Detection

The increasing scale and complexity of enterprise telemetry has fundamentally transformed cybersecurity from a rule-driven discipline into a data-centric problem. Modern Security Operations Centers (SOCs) generate millions of network events daily, making purely manual analysis operationally impossible. Consequently, machine learning has emerged as a foundational component of contemporary intrusion detection systems due to its ability to identify statistical deviations, classify attack patterns, and adapt to evolving behavioral trends. Current research broadly divides ML-driven cyber defense into two complementary paradigms: supervised categorical threat classification and unsupervised probabilistic anomaly detection.

2.3.1 Categorical threat classification and benchmarking

Supervised machine learning models remain highly effective for identifying known attack signatures when trained on representative network telemetry datasets. However, the effectiveness of these models is directly dependent on the realism and diversity of the training benchmark. Earlier datasets such as KDD Cup 1999 became increasingly obsolete due to

redundant traffic patterns and outdated attack behaviors, leading to inflated evaluation metrics that failed to generalize to real-world environments.

To address these shortcomings, Moustafa and Slay (2015) introduced the UNSW-NB15 dataset, a modern intrusion detection benchmark generated using the IXIA PerfectStorm framework. The dataset combines legitimate enterprise traffic with contemporary synthetic attacks spanning multiple categories including Exploits, Fuzzers, Generic attacks, Reconnaissance, and Denial-of-Service behaviors. Unlike legacy benchmarks, UNSW-NB15 preserves realistic protocol distributions and modern traffic characteristics, making it significantly more suitable for evaluating adaptive cyber defense architectures.

The complexity of such heterogeneous telemetry introduces substantial challenges for conventional classifiers because network events contain both numerical and categorical attributes with high cardinality. To overcome this limitation, Dorogush et al. (2017) proposed CatBoost, a gradient boosting framework specifically optimized for categorical feature handling. Unlike traditional boosting algorithms that require extensive preprocessing or one-hot encoding, CatBoost computes ordered target statistics through permutation-driven learning, thereby reducing target leakage and overfitting while preserving categorical semantics.

Within cybersecurity applications, CatBoost has demonstrated strong performance in intrusion classification tasks because features such as protocol type, service category, and connection state often carry significant discriminative power. Its probabilistic outputs also provide richer contextual information than rigid binary predictions, allowing downstream systems to reason about varying levels of confidence. Nevertheless, supervised classifiers remain fundamentally constrained by their training distribution. While highly effective at recognizing previously observed attack patterns, they struggle to generalize to zero-day exploits, distribution shifts, or adversarial behaviors absent from the training data.

2.3.2 Probabilistic anomaly detection

The inability of supervised models to reliably identify novel threats motivated a shift toward unsupervised anomaly detection techniques. Rather than learning explicit attack signatures, anomaly detection algorithms attempt to model the statistical structure of normal or expected behavior and identify events that deviate significantly from that baseline. This paradigm is particularly valuable in modern enterprise environments where attack patterns evolve faster than labeled datasets can be updated.

Among the most influential approaches is the Robust Random Cut Forest (RRCF) introduced by Guha et al. (2016). RRCF is specifically designed for high-dimensional streaming environments and operates by recursively partitioning feature space using random cuts. Anomalies are identified through collusive displacement (codisp), a metric that quantifies how strongly a data point perturbs the geometric structure of the forest. Unlike distance-based anomaly detectors, RRCF remains computationally efficient in high-volume telemetry streams and exhibits strong robustness to noisy or partially corrupted data.

Recent research further validates the operational relevance of probabilistic anomaly scoring in dynamic environments. Vashisth and Goyal (2024) demonstrated the effectiveness of RRCF-based detection within resource-constrained IoT ecosystems, highlighting its resilience against distribution shifts and evolving behavioral baselines. This capability is particularly important for Zero Trust environments, where trust must be continuously recalculated based on live telemetry rather than static authentication checkpoints.

A major advantage of anomaly detection systems is that they produce continuous probabilistic scores rather than deterministic labels. This allows cybersecurity platforms to reason about uncertainty and behavioral severity instead of reducing complex telemetry into simplistic “benign” or “malicious” categories. However, anomaly detection alone introduces its own limitations. Benign operational changes, software updates, or unusual but legitimate user behavior may produce high anomaly scores, leading to elevated false positive rates and analyst fatigue.

2.3.3 Meta-Learning for score fusion

The limitations of both supervised classification and unsupervised anomaly detection have motivated research into ensemble intelligence and meta-learning architectures. Supervised classifiers provide high precision for known threats but remain blind to novel attacks, whereas anomaly detection models excel at identifying deviations but often lack contextual specificity. Individually, each approach creates operational blind spots that reduce SOC reliability.

To address this challenge, Yang et al. (2023) surveyed the application of meta-learning techniques within cyberspace security and demonstrated that stacked ensemble architectures significantly improve generalization performance while reducing false positive rates. In such architectures, the outputs of multiple heterogeneous detection models are fused by a higher-level learner capable of reasoning across complementary risk signals.

Within cybersecurity, meta-learning enables systems to combine categorical threat probabilities with continuous anomaly scores to construct a unified risk representation. This fusion process is especially valuable in Zero Trust environments because access control decisions must operate on continuously evolving confidence estimates rather than isolated binary detections. By integrating probabilistic outputs from supervised and unsupervised models, meta-learners can capture both signature certainty and behavioral deviation simultaneously.

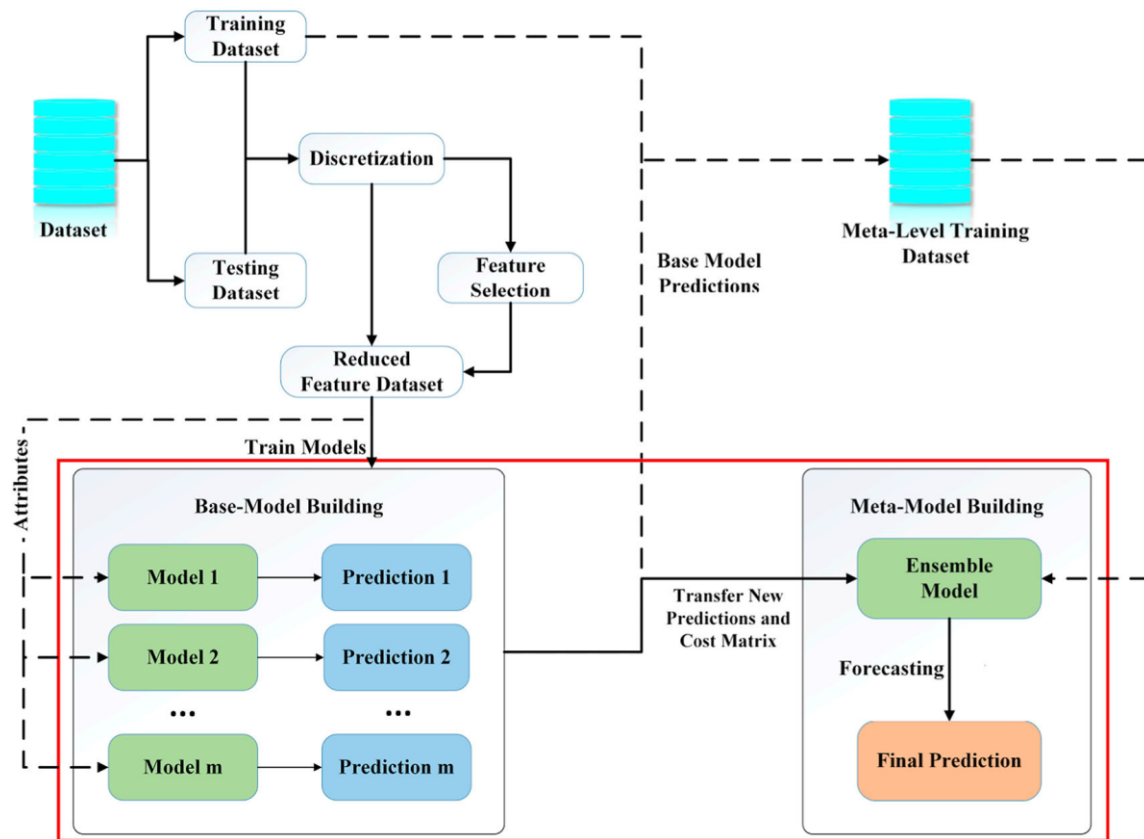


Fig 2.1: General meta-learning model based on stacked ensemble.[16]

2.4 Agentic AI in Cyber Defense

2.4.1 Static automation to autonomous workflows

The concept of utilizing cognitive architectures for complex problem solving traces back to foundational intelligence frameworks like the original SOAR cognitive architecture defined by Laird et al. (1987). In the contemporary cybersecurity landscape, Kshetri (2025) outlines how Agentic AI transforms defense mechanisms by enabling systems to combat emerging threats

autonomously. Unlike traditional static scripts, agentic frameworks utilize multi-layer reasoning. Roy and Singh (2026) proposed AgentSOC, a multi-layer framework demonstrating how autonomous agents can dynamically orchestrate security operations. Similarly, Ismail et al. (2025) advocate for a hyper-automation approach in SOCs, where Agentic AI processes continuous telemetry to execute context-aware responses rather than defaulting to hardcoded playbooks.

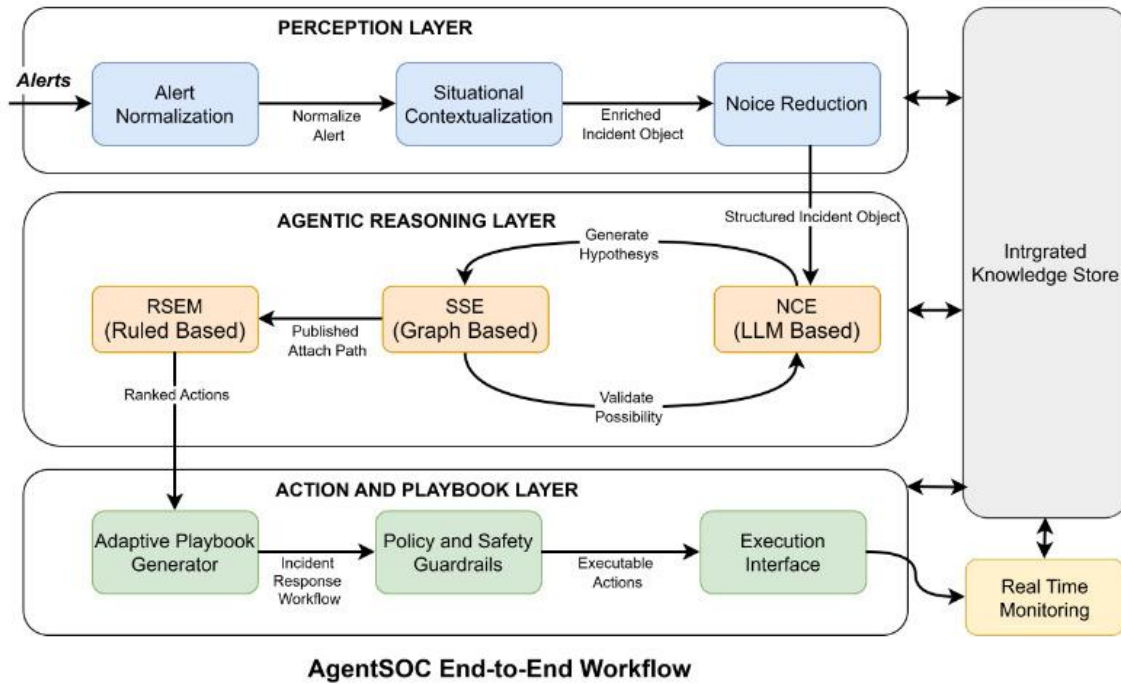


Fig 2.2: Proposed AgentSOC Architecture[14]

The pursuit of fully autonomous cyber defense is heavily supported by advancements in reinforcement learning (RL). Oesch et al. (2024) detail the operational path toward autonomous cyber defense, emphasizing the need for systems that can continuously adapt to adversarial tactics. Palmer et al. (2024) and Morris et al. (2025) extensively survey and evaluate the application of deep reinforcement learning agents in cyber defense, concluding that autonomous agents can effectively learn optimal containment strategies through continuous interaction with network environments.

2.4.2 Evaluating agentic security constraints

While Agentic AI offers transformative capabilities, it introduces new operational risks. Chhabra et al. (2025) comprehensively evaluate the threats, defenses, and open challenges of Agentic AI security. Entrusting autonomous agents, particularly those driven by generative LLMs with network isolation or firewall modification capabilities, presents severe risks of

"hallucinations" or catastrophic false positives. Consequently, the literature insists that agentic frameworks must be heavily constrained by deterministic policy guardrails to ensure safe and predictable execution.

2.5 Research Gap

A synthesis of contemporary literature reveals a significant architectural disconnect between modern machine learning-based threat detection and operational security automation frameworks. Existing Security Orchestration, Automation, and Response (SOAR) platforms have successfully improved incident response speed through orchestration and deterministic playbook execution; however, they remain fundamentally constrained by rigid, rule-based logic. Studies by Alliouche and Chenni (2025) and Ismail et al. (2025) emphasize that traditional SOAR systems struggle to adapt to uncertain or evolving attack behavior because they reduce complex telemetry into binary execution paths such as “allow,” “block,” or “escalate.” While effective for predefined signatures, these deterministic workflows are poorly suited for dynamic enterprise environments characterized by distribution shifts, behavioral ambiguity, and zero-day attack vectors.

Simultaneously, recent advances in machine learning have produced highly effective probabilistic detection models. Supervised algorithms such as CatBoost (Dorogush et al., 2017) demonstrate exceptional performance in categorical intrusion classification tasks, particularly when operating on heterogeneous network telemetry like the UNSW-NB15 dataset. Likewise, unsupervised approaches such as Robust Random Cut Forests (Guha et al., 2016; Vashisth and Goyal, 2024) provide strong capabilities for identifying behavioral deviations and novel anomalies through continuous probabilistic scoring. However, despite their statistical accuracy, these models operate primarily as isolated analytical systems. Existing research largely evaluates them in terms of classification metrics rather than their integration into real-time operational decision pipelines. Consequently, a critical gap persists between probabilistic machine learning outputs and actionable cybersecurity enforcement.

The literature further identifies that Zero Trust Architecture (ZTA) introduces additional operational complexity. Foundational works by Rose et al. (2020) and subsequent studies by Ojo (2025) establish that Zero Trust requires continuous trust re-evaluation rather than static, session-based authentication. In practice, however, many enterprise implementations still rely on fixed access policies that fail to adapt dynamically once access has been granted. Existing

automation tools lack the capability to continuously interpret evolving behavioral telemetry and translate it into proportional enforcement actions. As a result, there remains no widely adopted framework capable of reconciling probabilistic behavioral reasoning with deterministic Zero Trust policy constraints in real time.

Another major limitation identified throughout the literature is the absence of contextual reasoning and adaptive orchestration within existing SOC workflows. Current systems generally treat alerts as isolated events and fail to incorporate broader contextual signals such as anomaly severity, historical trust behavior, quarantine history, asset sensitivity, or uncertainty within the classification process itself. Although Yang et al. (2023) demonstrate that meta-learning and stacked ensemble architectures can significantly improve generalization and reduce false positive rates, current studies rarely operationalize these techniques within an end-to-end autonomous cyber defense pipeline. Most proposed frameworks remain conceptual or narrowly focused on isolated detection stages without addressing how probabilistic fusion outputs should guide downstream policy enforcement and remediation workflows.

Furthermore, while recent research increasingly advocates the integration of Agentic AI into Security Operations Centers (Ismail et al., 2025; Roy and Singh, 2026; Oesch et al., 2024), the majority of these works remain theoretical or high-level conceptual architectures. Existing studies discuss autonomous cyber defense, intelligent orchestration, and adaptive reasoning, yet there remains a lack of empirically validated frameworks demonstrating how autonomous agents can safely interact with probabilistic risk signals under strict operational guardrails. In particular, insufficient attention has been given to the challenge of balancing autonomous reasoning with deterministic safety constraints required in real-world enterprise environments. This creates a critical research gap in designing systems capable of combining adaptive intelligence with predictable, policy-compliant enforcement.

To address these limitations, this project proposes an Agentic SOAR framework that acts as a cognitive bridge between probabilistic machine learning and deterministic Zero Trust enforcement. The architecture integrates a cost-sensitive meta-learning fusion layer that combines categorical threat probabilities from CatBoost with behavioral anomaly scores generated by Robust Random Cut Forests. Rather than relying on isolated detections, the system constructs a unified probabilistic risk representation capable of reasoning under uncertainty. This fused risk signal is continuously evaluated by a deterministic Policy Agent

that computes a dynamic Trust Score using contextual organizational constraints including connection behavior, anomaly severity, and historical enforcement patterns.

Building upon this foundation, the framework introduces an LLM-driven multi-agent reasoning ecosystem capable of orchestrating graduated, context-aware enforcement actions rather than rigid binary responses. By integrating probabilistic anomaly scoring, ensemble intelligence, continuous trust evaluation, and autonomous playbook adaptation into a unified operational pipeline, the proposed architecture directly addresses the disconnect between modern ML-based threat detection and real-world cyber defense automation. In doing so, the framework aligns the statistical precision of contemporary machine learning with the operational requirements of adaptive, autonomous, Zero Trust-oriented cybersecurity systems.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Architecture Overview

The proposed framework replaces the static, rule-driven execution pipeline of traditional Security Operations Centers (SOCs) with a dynamic, closed-loop reasoning architecture. Rather than mapping alerts to fixed responses, the system continuously fuses probabilistic machine learning signals with deterministic Zero Trust policy constraints to orchestrate graduated, context-aware enforcement actions. The architecture operates across three distinct operational phases, each illustrated in the diagrams that follow.

Phase 1 — Offline ML Training Pipeline. Before deployment, an ensemble detection layer is trained offline on the UNSW-NB15 benchmark dataset. Raw network telemetry is preprocessed into two parallel feature paths: categorical features are fed directly to a CatBoost binary classifier and a multi-class incident agent, while numerical features are scaled and passed to a Robust Random Cut Forest (RCF) for spatial anomaly detection. To prevent data leakage, all base model predictions are generated using a five-fold Out-of-Fold (OOF) strategy. The resulting predictions : a categorical risk probability, an RCF anomaly score, a Shannon entropy scalar, and a top-2 class margin are fused by a cost-sensitive meta-learner into a single fused_risk score. A threshold optimization sweep then identifies the decision boundary that minimises operational SOC cost under a minimum precision constraint. All trained artifacts are serialised to disk and loaded at inference time.

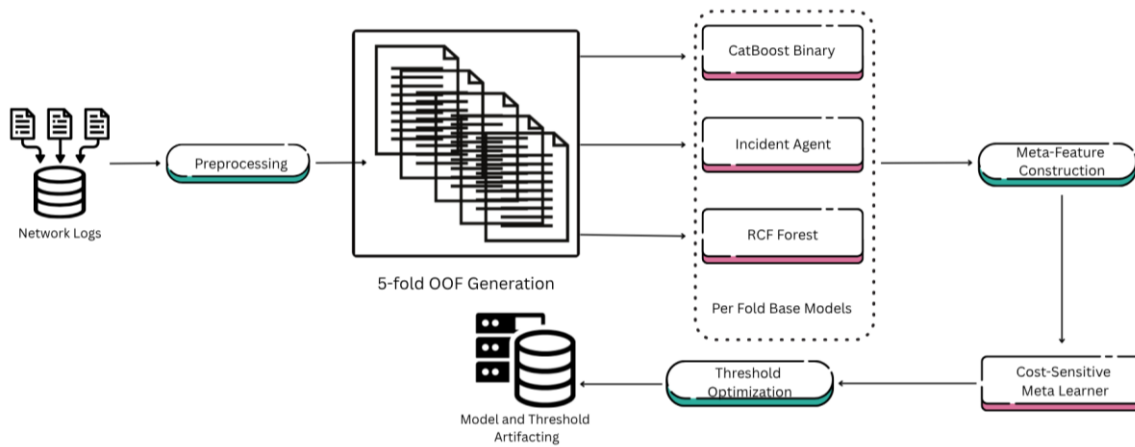


Fig 3.1: ML Training Pipeline

Phase 2 — SOAR Inference Pipeline. At runtime, each incoming network event is scored by the loaded ML artifacts to produce a fused_risk value. Events that fall below the optimised threshold are logged as benign and discarded without further computation. Events that exceed the threshold are escalated to the multi-agent reasoning layer. A deterministic Policy Agent evaluates the event against organisational constraints like connection volume, protocol whitelist, time-of-day policy, and historical quarantine status to produce a structured Trust Score and behaviour signature. If trust is acceptable and risk remains below a grey-zone ceiling, the event is fast-pathed to an ALLOW decision. Otherwise, a Large Language Model (LLM)-driven Response Agent analyses the slim event context against native routing rules and semantically retrieved learned rules from vector memory, returning a structured playbook decision. The selected playbook step/rule i.e, ALLOW, STEP_UP_AUTH, RATE_LIMIT_DOS, or NETWORK_ISOLATION, is then executed, and the resulting trust score and behaviour signature are persisted to agent memory.

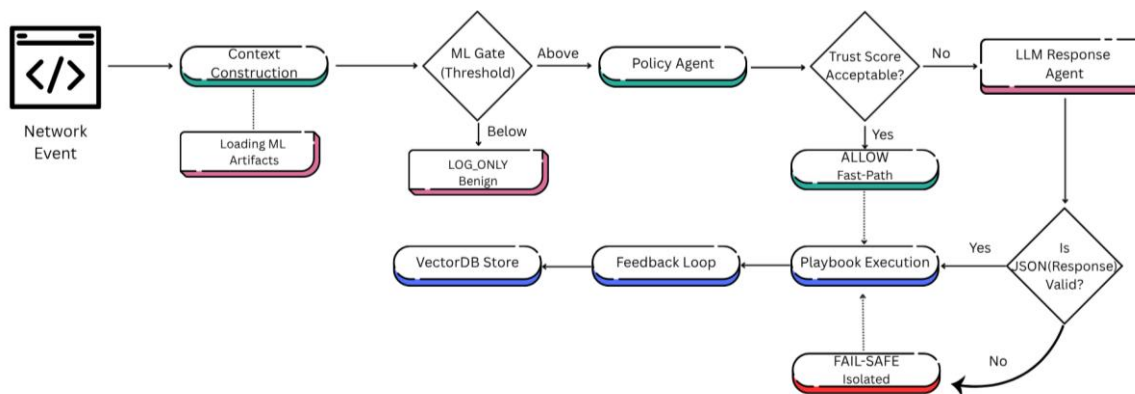


Fig 3.2: Agentic SOAR Pipeline

Phase 3 — Autonomous Self-Learning Loop. When ground-truth labels become available, the system compares them against its prior decisions to identify false positives and false negatives. Misclassified events are passed to the PlaybookEditorAgent, which invokes the LLM to perform a root-cause analysis of the failure. Provided the LLM is available and produces a valid analysis, a new IF/THEN routing rule is synthesised and appended to the active playbook. The rule is simultaneously embedded into a ChromaDB vector store, enabling semantic retrieval during future inference. A periodic consolidation step enforces a maximum rule count, merges redundant entries, and prunes stale rules to prevent playbook bloat.

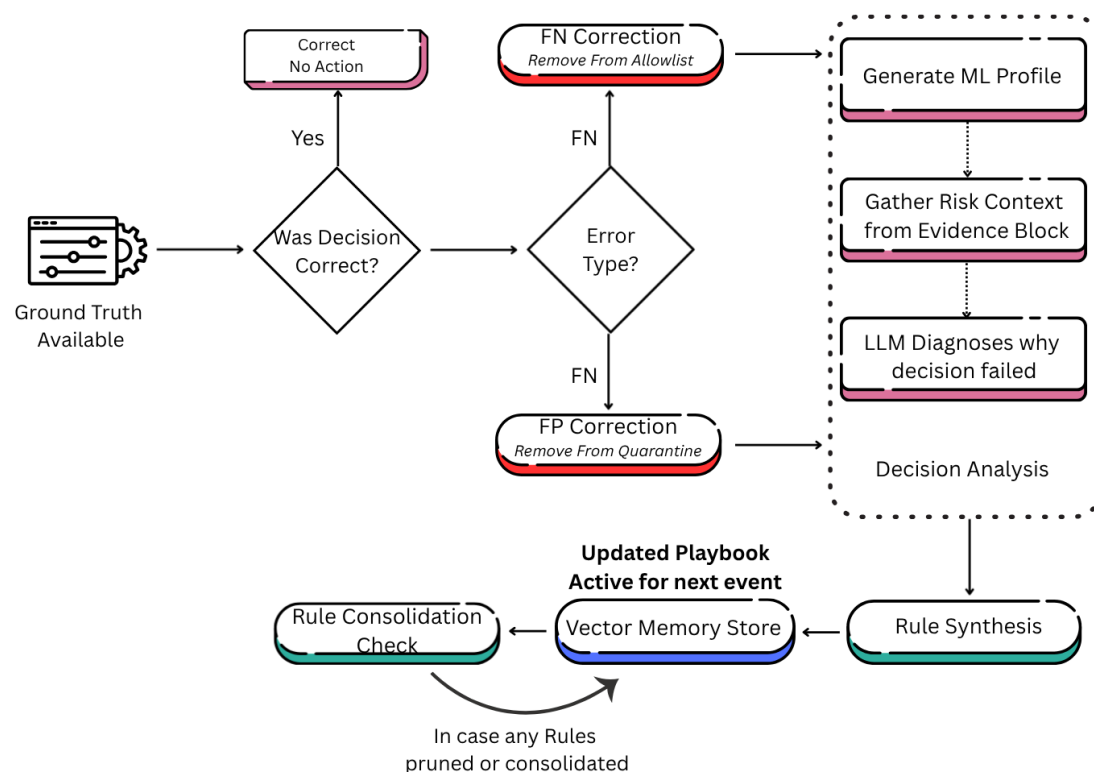


Fig 3.3: Self Learning Pipeline

Reconciling Probabilistic ML with Deterministic Enforcement. A central architectural challenge is reconciling the probabilistic nature of machine learning with the binary requirements of Zero Trust policy enforcement. ML models operate in gradients of confidence; security policies demand strict boundaries. The proposed architecture resolves this friction by positioning the Policy Agent as a deterministic guardrail between the ML fusion layer and the autonomous response agent. Rather than allowing the LLM to act directly on raw model scores, the Policy Agent translates fused_risk into a structured Trust Score by applying codified penalties for threat classification, quarantine history, asset sensitivity, and connection volume before any enforcement action is taken.

3.2 Dataset and Preprocessing

3.2.1 The UNSW-NB15 dataset

The experimental evaluation of this framework is conducted using the **UNSW-NB15** benchmark dataset, created by Moustafa and Slay[9]. The dataset was generated using the IXIA PerfectStorm traffic generator within a controlled network testbed, producing a hybrid of real modern normal traffic and synthetic contemporary attack behaviours across nine attack categories: DoS, Worms, Backdoors, Fuzzers, Generic, Reconnaissance, Shellcode, Exploits, and Analysis.

UNSW-NB15 was selected for this evaluation over legacy benchmarks such as KDD Cup 1999 for two reasons that directly align with this framework's objectives. First, its 37,000 normal traffic instances provide a sufficiently large and statistically clean baseline for the RCF forest to learn the normal behaviour manifold without attack contamination. Second, its severe class imbalance. Attack traffic constitutes roughly 64% of training samples and 68% of test samples faithfully replicating the alert fatigue conditions of operational SOC's, providing a realistic stress test for the cost-sensitive fusion layer. The full dataset statistics are summarised in Table 3.1.

SNo.	Name	Type	Description
36	is_sm_ips_ports	Binary	"If source (1) and destination (3)IP addresses equal and port numbers (2)(4) equal then this variable takes value 1 else 0"
39	is_ftp_login		If the ftp session is accessed by user and password then 1 else 0.
49	Label		0 for normal and 1 for attack records
7	dur	Float	Record total duration
15	Sload		Source bits per second
16	Dload		Destination bits per second
27	Sjit		Source jitter (mSec)
28	Djit		Destination jitter (mSec)
31	Sintpkt		Source interpacket arrival time (mSec)
32	Dintpkt		Destination interpacket arrival time (mSec)
33	tcprtt		"TCP connection setup round-trip time, the sum of synack and ackdat."
34	synack		"TCP connection setup time the time between the SYN and the SYN_ACK packets."
35	ackdat		"TCP connection setup time the time between the SYN_ACK and the ACK packets."
2	sport	Integer	Source port number
4	dsport		Destination port number
8	sbytes		Source to destination transaction bytes
9	dbytes		Destination to source transaction bytes
10	sttl		Source to destination time to live value
11	dttl		Destination to source time to live value
12	sloss		Source packets retransmitted or dropped
13	dloss		Destination packets retransmitted or dropped
17	Spkts		Source to destination packet count
18	Dpkts		Destination to source packet count
19	swin		Source TCP window advertisement value
20	dwin		Destination TCP window advertisement value
21	stcpb		Source TCP base sequence number
22	dtcpb		Destination TCP base sequence number
23	smeansz		Mean of the ?ow packet size transmitted by the src
24	dmeansz		Mean of the ?ow packet size transmitted by the dst
25	trans.depth		Represents the pipelined depth into the connection of http request/response transaction
26	res_bdy_len		Actual uncompressed content size of the data transferred from the servers http service.
37	ct_state_ttl		No. for each state (6) according to specific range of values for source/destination time to live (10) (11).
38	ct_flw_http_mthd		No. of flows that has methods such as Get and Post in http service.
40	ct_ftp_cmd		No of flows that has a command in ftp session.
41	ct_srv_src		No. of connections that contain the same service (14) and source address (1) in 100 connections according to the last time (26).
42	ct_srv_dst		No. of connections that contain the same service (14) and destination address (3) in 100 connections according to the last time (26).
43	ct_dst_ltm		No. of connections of the same destination address (3) in 100 connections according to the last time (26).
44	ct_src_ltm		No. of connections of the same source address (1) in 100 connections according to the last time (26).
45	ct_src_dport_ltm		No of connections of the same source address (1) and the destination port (4) in 100 connections according to the last time (26).
46	ct_dst_sport_ltm		No of connections of the same destination address (3) and the source port (2) in 100 connections according to the last time (26).
47	ct_dst_src_ltm		No of connections of the same source (1) and the destination (3) address in in 100 connections according to the last time (26).
1	srcip	Nominal	Source IP address
3	dstip		Destination IP address
5	proto		Transaction protocol
6	state		Indicates to the state and its dependent protocol
14	service		"http ftp smtp ssh dns ftp-data
48	attack_cat		"The name of each attack category. In this data set nine categories <i>eg</i> Fuzzers Analysis Backdoors DOS exploits Generic Reconnaissance Shellcode and Worms"
29	Stime	Timestamp	record start time
30	Ltime		record last time

Table 3.1: Features of UNSW-NB15 dataset[17]

The 49 raw features span five categories: 13 flow-level identifiers (source/destination IP, port, and protocol), 13 basic features (packet counts and byte transfer rates including spkts and dpkts), 8 content features (TCP sequence numbers and HTTP methods), 6 time features (jitter and inter-arrival durations), and 7 calculated behavioural features. Of these, features such as `ct_state_ttl` (which encodes deceptive TCP state-to-TTL mapping behaviours) are particularly discriminative for detecting evasion techniques that operate at the protocol level rather than the traffic volume level.

3.2.2 Feature engineering

Raw telemetry is split into three parallel feature representations at preprocessing time, as the detection models operate on fundamentally different views of the same data.

The categorical path isolates all object-typed columns alongside three numerically-coded categorical features `is_ftp_login`, `is_sm_ips_ports`, and `ct_state_ttl` that carry nominal rather than ordinal meaning. These columns are retained as strings and passed directly to the CatBoost-based models, which handle categorical encoding natively through ordered target statistics. Missing values are filled with the string 'unknown' to preserve categorical semantics without imputing a numerical value.

The numerical path processes the remaining continuous features through a two-stage pipeline. A `StandardScaler` first zero-centres and unit-normalises the feature matrix, after which PCA is applied with a 95% variance retention threshold. This dimensionality reduction serves two purposes: it reduces the computational overhead of the downstream meta-learner, and it decorrelates features that carry redundant signals. For example, `tcprrt`, `synack`, and `ackdat` all measure TCP round-trip timing at different granularities. The number of retained principal components is determined empirically by the variance threshold rather than fixed a priori, allowing it to adapt to the data distribution.

The RCF model operates on a third, independent representation. Rather than receiving the PCA-compressed feature space, the RCF is trained on raw numerical features scaled with a **RobustScaler** fitted exclusively on attack-labelled training rows. This design embeds two deliberate architectural choices. First, `RobustScaler`'s median/IQR-based normalisation is resistant to the extreme outliers present in UNSW-NB15 features such as `sbytes` and `res_bdy_len`, without requiring clipping. Second, and more fundamentally, the RCF is never

given PCA-transformed features because PCA rotation and compression destroys the sparse local geometric structure that isolation-based forests rely on to identify anomalies; codisp scores degrade when the original feature geometry is lost.

The consequence of training the RCF exclusively on attack traffic is that the forest learns the density manifold of attack behaviour rather than normal behaviour. A point that scores high codisp is therefore geometrically distant from the learned attack manifold meaning it resembles normal traffic. The raw RCF output is accordingly inverted ($1.0 - \text{score}$) before being passed to the meta-learner, so that the final anomaly signal remains semantically consistent: a high value indicates attack-like behaviour across all three base model outputs.

3.2.3 Preventing data leakage: OOF training

A critical implementation concern in any stacked ensemble is ensuring that the meta-learner is never trained on predictions generated by a model that has already seen the corresponding labels. If the base models are trained on the full training set and then used to generate meta-features on that same set, the resulting predictions will be artificially overconfident, and the meta-learner will learn to trust inflated scores that do not generalise to unseen data.

This framework addresses the leakage problem through a five-fold **Out-of-Fold (OOF)** strategy. The training set is partitioned into five stratified folds using StratifiedKFold with stratification on the multi-class label to preserve attack-class frequency across folds. For each fold, all three base models are trained exclusively on the remaining four folds and then scored against the held-out fifth fold. This produces a full set of OOF predictions that cover every training sample exactly once, with no sample ever scored by a model that was trained on it.

3.3 The ML Detection Layer

3.3.1 Binary threat classification with CatBoost

The primary detection signal is produced by a gradient-boosted decision tree classifier built on the CatBoost framework. CatBoost was selected over alternatives such as XGBoost or LightGBM for a specific architectural reason: its native handling of categorical features through ordered target statistics eliminates the need for one-hot or label encoding, which would artificially impose ordinal relationships on nominal features such as proto and service. This is

particularly important for UNSW-NB15, where protocol type and service category are among the strongest discriminators between normal and attack traffic.

The model is configured with 500 boosting iterations at a conservative learning rate of 0.05 and a maximum tree depth of 6. Heavy L2 leaf regularisation is applied to constrain model complexity and prevent overfitting on the training distribution, a necessary precaution given that UNSW-NB15's test set is drawn from a different time window than the training set, introducing a natural distribution shift. The model outputs a continuous probability score in $[0, 1]$ representing the likelihood that a given event is attack traffic. This score constitutes the first and most heavily weighted input to the meta-learner fusion layer.

A normal-class penalty mechanism is applied at inference time for the multi-class use case, subtracting a fixed offset of 0.15 from the predicted Normal class probability before taking the argmax. This asymmetric adjustment forces the model to require greater confidence before classifying a connection as benign, directly reducing the false negative rate on ambiguous attack traffic at the cost of a controlled increase in false positives. The penalty is applied only to the incident agent's multi-class output and does not affect the binary probability score passed to the meta-learner.

3.3.2 Multi-class incident agent and probability calibration

A second CatBoost model “the incident agent” is trained in parallel on the same categorical feature path to produce a fine-grained, per-class threat classification across all ten categories: Normal plus the nine UNSW-NB15 attack groups. Rather than binary cross-entropy, the incident agent uses the **MultiClassOneVsAll** loss function, which trains a separate binary classifier for each class and is better suited to the extreme class imbalance of the dataset than standard softmax. The evaluation metric is set to **TotalF1** to optimize for macro F1 across all attack classes, preventing the training signal from being dominated by the majority class.

Several additional regularisation measures are applied to improve generalisation on rare attack categories. **SqrtBalanced** automatic class weighting up-samples the gradient contribution of minority classes proportionally to the square root of their inverse frequency. A minimum of 20 samples per leaf (`min_data_in_leaf=20`) prevents the tree from constructing hyper-specific splits on rare-class instances, while elevated random strength (1.5) and bagging temperature (0.8) introduce controlled stochasticity to reduce variance.

The raw softmax outputs of a gradient-boosted classifier are not guaranteed to be well-calibrated probabilities; they tend to be overconfident, with predicted probabilities clustering near 0 and 1. This miscalibration would corrupt the Shannon entropy computation used as a meta-feature, since entropy is only a reliable uncertainty signal when the underlying probabilities are calibrated. To address this, the incident agent's outputs are post-hoc calibrated using isotonic regression via `CalibratedClassifierCV` with `cv='prefit'`, fitted on a dedicated 20% holdout split of the training data that the base model never sees. Isotonic regression is preferred over Platt scaling here because it is non-parametric and makes no assumptions about the shape of the calibration curve, which is appropriate given the multi-class, imbalanced setting. The resulting calibrated probability distribution across all ten classes is passed downstream to the meta-feature construction stage described in Section 3.4.1.

3.3.3 Anomaly scoring with robust random cut forest

The third detection signal is produced by a Robust Random Cut Forest (RRCF), an unsupervised streaming anomaly detection algorithm introduced by Guha et al. (2016). Unlike the CatBoost models, which learn explicit decision boundaries from labelled data, the RCF assigns anomaly scores based on how much a point disrupts the geometric structure of a random ensemble of binary space-partitioning trees. A point that causes a large change in the **collusive displacement (codisp)** of the forest is considered anomalous. This makes the RCF sensitive to zero-day and novel attack patterns that fall outside the known attack categories present in the training labels, precisely the traffic that a purely supervised model is structurally blind to.

The forest is configured with 80 trees and a maximum leaf size of 256, balancing anomaly sensitivity against computational cost. As described in Section 3.2.2, the RCF is trained exclusively on attack-labelled rows using an attack-trained architecture: by learning the density manifold of known attack traffic rather than normal traffic, the forest identifies as anomalous any point that is geometrically distant from that manifold — which in practice corresponds to normal traffic. The raw codisp output is therefore inverted before downstream use.

Score normalisation uses a percentile-anchored linear stretch rather than a sigmoid transform. After training, the 1st and 99.5th percentiles of the raw codisp distribution are computed and stored as calibration anchors. At inference time, raw codisp scores are mapped to the normalized range $[0,1]$ via :

$$normalized = clip\left(\frac{raw-p_1}{p_{99.5}-p_1}, 0, 1\right) \quad (\text{eq 3.1})$$

Using the 99.5th percentile as the effective ceiling rather than a fixed multiplier on the 99th percentile (also proven experimentally) ensures the ceiling adapts to the actual tail shape of the score distribution. By construction, exactly 0.5% of training scores will saturate at 1.0, making saturation predictable and diagnosable. A saturation rate above 1% at inference time is treated as a distribution shift warning.

During the OOF generation phase, the calibration anchors computed on the full training set are propagated into each fold's temporary RCF instance, ensuring all five folds share the same score scale. This prevents the meta-learner from receiving OOF anomaly scores that are normalised on inconsistent scales across folds, which would otherwise introduce a systematic source of bias into the fusion layer.

3.4 The Cost-Sensitive Meta-Learner (Fusion Layer)

3.4.1 Meta-Feature construction: Shannon entropy and top-2 margin

The three base models described in Section 3.3 each produce outputs of different types: a scalar binary probability from CatBoost, a normalised anomaly score from the RCF, and a ten-dimensional calibrated softmax distribution from the incident agent. The meta-learner requires a fixed-width input vector, so the incident agent's output must be reduced to a compact representation without discarding the uncertainty information it carries.

Feeding all ten class probabilities directly into the meta-learner would be counterproductive. A logistic regression model with ten additional inputs trained on roughly 82,000 samples risks overfitting to the specific class frequency distribution of the training set, and would generalise poorly to traffic distributions where the relative frequency of attack categories shifts which is a common occurrence in real SOC environments. Instead, the incident agent's output is reduced to two scalar meta-features that capture the structure of the distribution without encoding its specific class assignments.

The first is Shannon entropy, computed over the calibrated class probability vector as:

$$H = - \sum_k p_k \log p_k \quad (\text{eq 3.2})$$

High entropy indicates the incident agent is uncertain across multiple attack categories - a signal that the event occupies an ambiguous region of the feature space that the binary CatBoost score alone may underweight. Low entropy indicates the agent is confident about a single class, whether Normal or a specific attack type, and the binary score carries sufficient information.

The second is the **top-2 margin**, defined as the difference between the highest and second-highest class probabilities. Where entropy captures global distributional uncertainty, the top-2 margin captures local ambiguity: a narrow margin between the top two classes indicates the agent is genuinely split between two specific interpretations of the event, while a wide margin indicates a dominant, unambiguous classification. The two scalars are complementary, an event can have moderate entropy but a very narrow top-2 margin if probability mass is spread unevenly across several classes, and the two signals together characterise that structure more precisely than either alone.

The final meta-feature matrix presented to the fusion layer is therefore a four-column array per sample: CatBoost binary probability, RCF anomaly score, Shannon entropy, and top-2 margin. The fusion layer models the probability of malicious activity using an L2-regularized logistic regression classifier operating on the four-dimensional meta-feature vector. The fused risk score is computed as:

$$fused_risk = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b) \quad (\text{eq 3.3})$$

where:

- x_1 represents the CatBoost binary probability,
- x_2 represents the RCF anomaly score,
- x_3 represents Shannon entropy,
- x_4 represents the top-2 class margin,
- w_i are the learned feature weights,
- b is the bias term,
- and σ is the sigmoid activation function defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (\text{eq 3.4})$$

The resulting fused_riskscalar represents a unified probabilistic estimate of malicious activity that combines categorical certainty, behavioral anomaly deviation, and classification uncertainty into a single decision signal suitable for downstream Zero Trust policy evaluation.

3.4.2 Cost sensitive training and threshold optimisation

The meta-learner is implemented as a logistic regression model with L2 regularisation, trained on the clean OOF meta-feature matrix described in Section 3.2.3. The use of logistic regression rather than a more complex model is deliberate: with only four input features and a well-defined binary output, a linear decision boundary is sufficient, and the resulting weight vector is directly interpretable where each weight reflects the marginal contribution of its corresponding base model signal to the final risk score.

Standard logistic regression treats false positives and false negatives as equally costly. In a SOC context this assumption is operationally incorrect: a missed attack (false negative) exposes the organisation to a security breach, while an unjustified alert (false positive) contributes to analyst fatigue but does not itself cause harm. The meta-learner encodes this asymmetry directly into the gradient update by weighting each sample's error term according to its ground truth label. Attack samples incur a penalty of 12 and normal samples a penalty of 2, producing a 6:1 cost ratio that keeps the model under continuous recall pressure throughout training. The gradient update for weights $\mathbf{w}$ at each epoch is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{n} \mathbf{X}^T [(\hat{y} - y) \odot \mathbf{c}] + \frac{\lambda}{n} \mathbf{w} \quad (\text{eq 3.5})$$

where,

- $\mathbf{c}$ is the per-sample cost vector
- λ is the L2 regularisation coefficient
- n is the number of samples
- $\mathbf{X}$ is the feature matrix
- $\mathbf{w}$ is the weight matrix

The second term penalises large weights to prevent any single base model from dominating the fusion. A diagnostic check verifies that CatBoost's learned weight accounts for at least 30% of the total absolute weight mass; if it falls below this floor, the meta-learner is considered to be over-relying on the RCF or entropy signals, and the cost ratio is adjusted accordingly.

The model is trained using gradient descent with early stopping: if the maximum absolute gradient across all weights and the bias falls below $\epsilon = 1\text{E-}5$, training terminates early and the converged weights are retained. A StandardScaler is fitted on the OOF meta-feature matrix and stored inside the model artifact, ensuring that the same normalisation is applied at inference time without requiring an external scaler file.

Once trained, the meta-learner produces a continuous fused_risk score in $[0, 1]$ for every training sample. Rather than applying a fixed 0.5 decision threshold, an optimisation sweep selects the threshold that minimises the same asymmetric SOC cost function used during training. One thousand candidate thresholds are evaluated uniformly across $[0.01, 0.99]$, and each is assessed against two simultaneous constraints: precision must be at least 0.75, and recall must be at least 0.83. The precision floor ensures the system does not generate an operationally unsustainable volume of false alerts, while the recall floor guarantees that at minimum 83% of actual attacks are caught regardless of how aggressively the precision constraint pushes the threshold upward. The threshold satisfying both constraints at minimum SOC cost is serialised to disk and loaded at inference time, ensuring the live system operates at exactly the boundary validated on the OOF distribution.

3.5 The Zero Trust Policy Agent

3.5.1 Trust score computation

Events that exceed the ML detection threshold are escalated to the Policy Agent, a fully deterministic rule engine that evaluates each event against a set of codified organisational constraints and produces a structured Trust Score. The policy agent is deliberately separated from the ML layer: its role is not to re-classify the event but to contextualise it — translating the probabilistic fused_risk into an interpretable, auditable score that reflects the cumulative weight of multiple independent risk signals.

The trust score is initialised at 1.0 (full trust) and reduced additively by a series of penalty factors. This design means each factor's contribution to the final score is explicit and individually logged, producing a transparent audit trail for every enforcement decision. A simplified representation of the trust computation process is defined as:

$$TrustScore = 1.0 - (w_1R + w_2Q + w_3A + w_4C + \dots) \quad (\text{eq 3.6})$$

where:

- R represents the fused probabilistic risk generated by the meta-learner,
- Q represents quarantine or memory-based penalties,
- A represents asset sensitivity amplification,
- C represents anomalous connection-volume behavior,
- and w_i denote configurable policy weights associated with each risk factor.

The ellipsis indicates additional deterministic policy penalties applied by the rule engine, including protocol violations, after-hours access, and categorical threat escalations.

Seven penalty factors are evaluated in sequence:

The **ML fused risk penalty** is computed as $\text{fused_risk} \times \text{ml_risk_weight}$, where ml_risk_weight is set to 0.30. Capping the ML contribution at 30% of the initial trust score is an intentional design constraint: it prevents a single high-confidence ML score from collapsing the trust score to near-zero before the deterministic policy signals have been applied, ensuring that all factors contribute meaningfully to the final decision. For events destined for sensitive network subnets, this penalty is amplified by an asset sensitivity multiplier so traffic targeting critical infrastructure prefixes (e.g. 10.0.0.x) incurs a $1.5\times$ amplification of the ML penalty only, not of the total accumulated deficit.

The **threat classification penalty** maps the incident agent's predicted attack category to a fixed penalty value configured in the policy configuration. Severity tiers range from 0.00 for Normal traffic to 0.45 for Shellcode, reflecting the operational risk associated with each attack class. Any threat class with a penalty exceeding 0.35 is additionally recorded as a policy violation.

The **memory signal** applies a quarantine penalty of 0.40 if the event's behaviour signature has previously been confirmed as malicious, or an allowlist bonus of 0.15 if it has been verified as benign. For signatures in the intermediate monitoring state where evidence has accumulated

but not yet reached a promotion or demotion threshold, a continuous `trust_bias` value is applied, providing a soft, graded signal rather than a binary jump.

The remaining four factors apply fixed penalties for specific policy violations: an **unapproved protocol/service combination** (any `proto_service` pair not on the organisational whitelist) incurs a 0.10 penalty; **after-hours access** (connections outside configured business hours) incurs 0.10; a **high connection volume** exceeding 5,000 source packets (`spkts > 5000`) incurs 0.10; and a **categorical risk spike** where the CatBoost probability alone exceeds 0.70 incurs a further 0.10. The final score is clamped to `[0.0, 1.0]`.

If the resulting trust score meets or exceeds the configured floor of 0.40 and the `fused_risk` falls below a grey-zone ceiling of 0.85, the event is fast-pathed to an ALLOW decision without LLM involvement. Events that fail either condition are escalated to the response agent described in Section 3.6.

3.5.2 Behaviour signatures and the evidence ledger

Every event evaluated by the Policy Agent is assigned a behaviour signature — a structured string key that serves as the persistent identity record for that traffic pattern across sessions. The signature encodes eight fields derived from the event's telemetry and ML outputs:

`proto_service_state_threatclass_RB_AB_VB_EB`

where `proto`, `service`, and `state` are raw telemetry fields identifying the connection's transport and application layer characteristics; `threatclass` is the incident agent's predicted attack category; and `RB`, `AB`, `VB`, `EB` are four bucketed ML signal codes. The risk bucket (`R1–R4`) encodes the fused risk tier: `R1` below 0.40, `R2` up to 0.65, `R3` up to 0.85, and `R4` above 0.85. The anomaly bucket (`A1–A3`) encodes the RCF score tier: `A1` below 0.40, `A2` up to 0.70, and `A3` above. The volume bucket (`V1–V3`) encodes source packet count bands: `V1` below 100, `V2` up to 5,000, and `V3` above. The entropy bucket (`E1–E3`) encodes classification certainty: `E1` below 0.5 (high certainty), `E2` up to 1.5, and `E3` above (high uncertainty). Including the ML signal buckets in the signature means that the same protocol and service combination is tracked as a distinct identity at different risk levels. For example: a TCP/HTTP Normal `R1` connection and a TCP/HTTP Generic `R3` connection are separate signatures with separate evidence histories.

Each signature is tracked in a persistent evidence ledger stored in `agent_memory`. The ledger records, for each signature, the accumulated count of confirmed normal verdicts, confirmed attack verdicts, the current status (monitoring, allowlisted, or quarantined), and a continuous `trust_bias` value. When ground truth becomes available — either through analyst confirmation or the self-healing loop, `update_signature_evidence` is called with the verified verdict, and the ledger is updated according to the following promotion and demotion rules.

A signature in monitoring status accumulates evidence with each confirmed verdict. Normal verdicts increment the `confirmed_normal` counter and nudge the `trust_bias` upward by a fixed step (capped at +0.20); attack verdicts increment `confirmed_attack` and nudge `trust_bias` downward (floored at -0.30). Once a signature reaches the threshold of confirmed normal verdicts(3 in our testing) with zero confirmed attacks, it is promoted to allowlisted status and added to the dynamic allowlist, granting it a +0.15 trust bonus on future events. Once it accumulates min threshold confirmed attacks(2 in our testing) with zero confirmed normals, it is promoted to quarantine status, adding a -0.40 penalty on future events. These thresholds are intentionally asymmetric: allowlisting requires more evidence than quarantining, reflecting the higher cost of missing a true attack relative to the cost of delaying a legitimate connection.

Promotion decisions are not permanent. If a quarantined signature subsequently accumulates necessary confirmed normal verdicts, it is demoted back to monitoring status and removed from the active quarantine list. Symmetrically, if an allowlisted signature accumulates the required number of confirmed attack verdicts, it is demoted and removed from the allowlist. This bidirectional evidence model prevents the ledger from becoming stale under concept drift, a traffic pattern that changes behaviour over time will gradually accumulate counter-evidence and transition to an appropriate status without manual intervention.

3.6 The LLM-Driven Response Agent

3.6.1 Prompt engineering and context construction

Events that fail the Zero Trust evaluation are passed to the LLM-driven response agent, which selects an appropriate enforcement action based on a structured analysis of the event. The agent is backed by DeepSeek-R1:8B, running locally, with temperature set to 0.0 to ensure deterministic, auditable decisions sampling-induced randomness is incompatible with a SOC environment where the same input must always produce the same enforcement outcome.

Rather than presenting the model with the full raw telemetry and all ML scores, a slim event context is constructed containing only the fields that the routing rules actually reference: the transport protocol, service, connection state, packet counts, threat classification, and fused risk score. Passing the full telemetry row would introduce irrelevant noise and increase the risk of the model anchoring on spurious correlations. Alongside the raw values, four semantic severity labels are derived from the ML scores and included in the context — mapping the fused risk, anomaly score, traffic volume, and classification certainty each to a human-readable tier such as LOW, MEDIUM, HIGH, or CRITICAL. These labels ensure the model reasons about risk in operationally meaningful terms rather than attempting to interpret raw decimal probabilities.

The system prompt is assembled from four distinct components, each serving a specific role in guiding the model toward a consistent, rule-grounded decision:

Behavioural history block: Included only when the event's behaviour signature has an established record in the evidence ledger. It summarises the signature's prior confirmed verdicts, current trust status, and accumulated bias.

Native routing rules block: This presents a numbered set of IF/THEN enforcement rules in priority order (avoid ambiguity), with the decision thresholds substituted numerically. These thresholds are derived directly from the OOF-optimised value produced during training, ensuring the LLM's rule boundaries remain consistent with the ML layer's calibration..

Learned rules block: Conditionally included when semantically similar past decisions exist in the vector memory store. Up to two prior decisions are retrieved by cosine similarity and injected only if they exceed a minimum relevance threshold.

Few-shot examples: These anchor the expected reasoning style and output format, reducing the likelihood of the model producing verbose or malformed responses when the routing rules leave room for interpretation.

The user prompt itself is simply the slim event context serialised as a structured data object. The model is instructed to return a single JSON object containing a brief reasoning statement, a playbook selection from the four defined options, and a boolean false positive flag with no additional text or formatting.

If the model returns a valid response, the decision and its reasoning are embedded into the vector memory store for future retrieval. If the model times out, returns unparseable output, or raises any error, a fail-safe of immediate network isolation is applied. The fail-safe defaults to isolation rather than allow, consistent with the cost asymmetry established throughout the pipeline. An unresolved uncertainty is treated as a threat rather than as benign.

3.6.2 SOAR playbook execution

It should be noted that the SOAR playbooks implemented in this framework are generic representations constructed for research purposes. Production-grade SOAR playbooks are proprietary to commercial vendors such as Splunk SOAR, Palo Alto XSOAR, and Microsoft Sentinel, and are not publicly available for academic use. The playbooks defined here are therefore designed to reflect the structure and intent of real enterprise playbooks referencing realistic tooling such as firewall APIs, EDR platforms, and identity management systems while acknowledging that the underlying integrations are simulated rather than live.

The selected playbook maps to a structured action sequence defining the specific tools and enforcement steps to be executed. Four active playbooks are defined, each representing a distinct enforcement posture calibrated to a threat severity tier.

ALLOW is the lowest enforcement posture, applied to events classified as benign or as false positives. The event is logged to the SIEM for audit purposes and no network change is made.

STEP_UP_AUTH is applied to ambiguous events where risk is elevated but the threat classification does not confirm a specific attack type. A push MFA challenge is issued to the connecting identity, a medium-severity alert is created for analyst review, and a Tier-1 SOC ticket is opened.

RATE_LIMIT_DOS is applied when the incident agent identifies DoS traffic. Traffic from the source is throttled rather than blocked outright, because volumetric attacks frequently involve distributed or spoofed sources where a single-IP block provides limited mitigation and risks disrupting legitimate traffic.

NETWORK_ISOLATION is the highest enforcement posture, applied to confirmed malicious signatures, critical-risk events, pre-quarantined signatures, and all LLM fail-safe responses. Both network-layer and endpoint-layer enforcement are triggered simultaneously-

the source IP is blocked at the firewall, the host is isolated at the EDR level, active session tokens are revoked, and the on-call engineer is paged at critical priority.

3.7 Autonomous Self-Healing Loop

3.7.1 Vector memory architecture

The self-learning loop relies on a semantic vector memory store that persists three categories of information across sessions: incident summaries from past LLM decisions, root-cause correction analyses from detected misclassifications, and synthesised routing rules produced by the agent. When a misclassification is detected, the system retrieves semantically similar past corrections from this store to inform the current analysis, ensuring the LLM reasons in the context of patterns it has encountered before rather than treating each correction in isolation. The vector store is queried by cosine similarity and a distance threshold is enforced so that only genuinely relevant past cases are surfaced. This infrastructure is standard and not a contribution of this work; the contribution lies in how the `PlaybookEditorAgent` uses it.

The Loop is triggered whenever ground truth becomes available for an event that was previously evaluated by the SOAR pipeline. It compares the system's decision against the verified label and, upon detecting a mismatch, initiates an autonomous correction cycle. Corrections arising from LLM fail-safe responses are explicitly excluded from this cycle, as they carry no diagnostic signal and anchoring a rule to a timeout event would be unsound.

3.7.2 False positive correction

When a ground-truth normal event was wrongly escalated, the agent initiates a two-stage LLM interaction. The first stage is a root-cause diagnosis. The system prompt frames the LLM as an internal cognitive auditor that has detected a false positive, and instructs it to produce a concise technical explanation of why the escalation occurred. The prompt directs the model to consider which ML signal was artificially elevated, which policy penalties contributed disproportionately to the trust score collapse, and what specific traffic feature - protocol, connection state, or packet asymmetry may have confused the models. If semantically similar past corrections exist in vector memory, they are prepended to the prompt as reference cases.

The full ML profile, semantic severity labels, trust factor breakdown, and key telemetry fields are provided as the user input, and the model is instructed to return plain text only.

```
user_prompt = (
    f"{{prior_ctx}}"
    f"{{evidence_block}}"
    f"AI decision : {{decision.get('playbook')}}\n"
    f"AI reasoning : {{decision.get('reasoning', 'none')}}\n"
    f"ML profile : {{json.dumps(ml_profile)}}\n"
    f"Risk context :\n"
    f" - Fused risk severity : {{semantic['fused_risk_severity']}}\n"
    f" - Anomaly severity : {{semantic['anomaly_severity']}}\n"
    f" - Traffic volume : {{semantic['traffic_volume']}}\n"
    f" - Classification certainty : {{semantic['classification_certainty']}}\n"
    f"Trust factors : {{json.dumps(trust_factors)}}\n"
    f"Telemetry : {{json.dumps({k: telemetry.get(k) for k in ['proto', 'service', 'state', 'spkts', 'dpkts']})}}\n"
    f"Threat class : {{context.get('predicted_threat_classification')}}\n\n"
    f"Diagnose the root cause of this autonomous False Positive."
)
```

Fig 3.4: Analysis Prompt

The second stage uses this diagnosis as the foundation for rule synthesis. A separate prompt frames the LLM as a playbook auditor and instructs it to write a single JSON routing rule that would correctly ALLOW similar benign events in future. The rule must anchor its condition to at least two signal features from the event telemetry, formatted as plain AND-separated key-value pairs. The diagnosis from the first stage, the behavioural history of the signature, and the event's ML and telemetry context are all provided. The model returns a structured rule object containing a condition, a playbook assignment of ALLOW, a boolean false positive flag, and a brief rationale. This rule is immediately written to the active playbook, embedded into vector memory, and the event's behaviour signature is submitted to the evidence ledger as a confirmed normal verdict.

3.7.3 False negative correction

When a ground-truth attack was wrongly allowed, the correction cycle follows the same two-stage structure but with materially different prompt framing. The diagnosis prompt identifies this as a critical security failure and directs the LLM to consider whether the fused risk was too low despite real attack activity, whether a trust bonus such as an established allowlist entry overrode a risk signal, or whether specific traffic characteristics disguised the attack from the detection layer. The synthesis prompt then instructs the model to write a NETWORK_ISOLATION rule rather than an ALLOW rule, anchoring the condition to the features that made the attack difficult to detect. Additionally, the LLM's reasoning is parsed to identify which existing routing rule was cited as responsible for the missed detection, and that

rule's false negative counter is incremented in a persistent rule health tracker. Once a rule accumulates two false negatives it is flagged for human review, surfacing systematically weak rules that the autonomous loop cannot safely fix on its own.

3.7.4 Rule insertion and deduplication

Newly synthesised rules are inserted at the top of the routing rule list, giving them priority over older and more general logic. Before insertion, the new rule's condition is checked against all existing learned rules for overlap: if an existing rule already covers the same traffic pattern with fewer clauses, the new rule is considered redundant and discarded. If the new rule is broader than an existing one, it replaces it. Rules that conflict in playbook direction (ALLOW versus NETWORK_ISOLATION) are always both retained and flagged for human review, since conflicting rules on the same traffic pattern represent a genuine ambiguity that requires analyst judgement.

3.7.5 Periodic consolidation

Every ten events, a consolidation pass is triggered. The LLM is presented with all currently active learned rules and instructed to identify near-duplicates rules that share the same protocol, connection state, and threat class etc. but differ only in minor telemetry values and merge them into broader generalisable rules that drop the overly specific clauses. Rules that differ in playbook direction are never merged. The consolidated rule set replaces the pre-consolidation rules in the active playbook, and the vector memory store is synchronised accordingly: embeddings for superseded rules are deleted and the consolidated rules are re-embedded. A maximum rule count ceiling prevents unbounded growth, with the oldest learned rules compacted out when the ceiling is reached. Native routing rules defined at system initialisation are never touched by the consolidation process.

CHAPTER 4

RESULTS AND ANALYSIS

4.1 Experimental Setup and Evaluation Methodology

The experimental evaluation is conducted in two distinct phases, reflecting the two operational stages of the framework. The first phase evaluates the ML detection pipeline in isolation on the full UNSW-NB15 test set of 175,341 samples, providing a controlled assessment of each component's contribution to detection performance. The second phase evaluates the complete agentic pipeline including the Policy Agent, LLM Response Agent, and PlaybookEditorAgent on a stratified sample of 100 events drawn from the test set, constrained by the computational cost of local LLM inference.

The train/test split follows the official UNSW-NB15 partition: 82,332 samples for training and 175,341 for evaluation, drawn from separate time windows to introduce a natural distribution shift between training and deployment conditions. This shift is intentional, it stress-tests the generalisation of models trained on one traffic period against a different one, more faithfully simulating real SOC deployment than a random split would. The class distribution is heavily imbalanced in both partitions: attack traffic constitutes 55% of training samples and 68% of test samples, reflecting the alert-heavy environment the cost-sensitive pipeline is designed for.

All base models are trained using the five-fold Out-of-Fold strategy described in Section 3.2.3, ensuring the meta-learner is trained on predictions that are statistically independent of the labels used to produce them. The threshold optimization sweep is performed on the OOF predictions rather than the test set, meaning the final decision boundary is selected entirely without exposure to held-out data. Test set evaluation is therefore a true holdout, no hyperparameter, threshold, or calibration decision is informed by it.

The cost function used throughout evaluation is asymmetric, consistent with SOC operational priorities: false negatives carry a penalty of 12 and false positives a penalty of 2, producing a 6:1 ratio that keeps missed attacks as the primary optimisation target. All reported SOC cost figures use this function. Standard classification metrics - precision, recall, F1, and accuracy are reported alongside confusion matrices to provide a complete picture of performance across both classes. For the agentic evaluation, metrics are drawn from the post-run analytics produced

after the 100-event SOAR session, and individual event traces are used to illustrate specific decision behaviours where relevant.

4.1.1 Evaluation metrics

The proposed framework is evaluated using a collection of supervised classification metrics designed to measure both predictive performance and operational suitability within a Security Operations Center (SOC) environment. Because the UNSW-NB15 dataset exhibits severe class imbalance and contains multiple attack categories with varying frequencies, relying on a single metric such as accuracy would provide an incomplete and potentially misleading assessment of model behavior. Consequently, this evaluation incorporates precision, recall, F1-score, weighted and macro averages, accuracy, and Receiver Operating Characteristic–Area Under Curve (ROC-AUC) to provide a comprehensive analysis of detection effectiveness.

Accuracy

Accuracy measures the proportion of total predictions that are correctly classified. It is defined as the ratio of correctly predicted samples to the total number of evaluated samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (\text{eq. 4.1})$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

Although accuracy provides a high-level overview of classifier performance, it becomes unreliable in highly imbalanced cybersecurity datasets. In SOC environments, benign traffic often dominates the dataset distribution, meaning a model may achieve high accuracy while still failing to detect genuine attacks effectively.

Precision

Precision measures the proportion of predicted attack events that are actually malicious:

$$Precision = \frac{TP}{TP + FP} \quad (\text{eq. 4.2})$$

High precision indicates that the model generates fewer false positives, which is particularly important in cybersecurity operations because excessive false alerts contribute directly to analyst fatigue and operational inefficiency.

Recall

Recall, also known as sensitivity or detection rate, measures the proportion of actual malicious events that are correctly identified:

$$Recall = \frac{TP}{TP + FN} \quad (\text{eq 4.3})$$

Within intrusion detection systems, recall is often considered more critical than precision because false negatives correspond to undetected attacks. A low recall value implies that malicious traffic is bypassing the detection pipeline, potentially leading to severe security breaches.

F1-Score

The F1-score combines precision and recall into a single harmonic mean, balancing the trade-off between minimizing false positives and minimizing false negatives:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (\text{eq 4.4})$$

The harmonic mean penalizes extreme imbalances between precision and recall, making the F1-score particularly useful for cybersecurity datasets where both alert quality and attack detection capability are operationally important.

Weighted F1

The weighted F1-score is computed as the support-weighted mean of the individual class F1-scores:

$$Weighted\ F1 = \sum_{i=1}^n w_i \times F1_i \quad (\text{eq 4.5})$$

Unlike macro averaging, the weighted F1-score accounts for class imbalance by assigning greater influence to classes with higher sample frequencies, making it particularly suitable for evaluating intrusion detection systems operating on imbalanced SOC telemetry datasets such as **UNSW-NB15**.

Macro Average

Macro averaging computes the arithmetic mean of a metric independently across all classes without considering class frequency. Each class contributes equally to the final score regardless of how many samples belong to that category. This evaluation is especially important for multi-class intrusion detection because rare attack categories such as Worms or Shellcode should not be overshadowed by dominant traffic classes.

Macro averages therefore evaluate how consistently the classifier performs across all attack types and provide insight into generalization capability under class imbalance conditions.

Weighted Average

Weighted averaging computes the mean of a metric across all classes while weighting each class according to its sample frequency:

$$\text{Weighted Average} = \sum_{i=1}^n w_i \times \text{Metric}_i \quad (\text{eq 4.6})$$

where w_i represents the proportion of samples belonging to class i .

Unlike macro averaging, weighted averages emphasize performance on dominant classes. This metric is particularly useful for estimating real-world operational performance because enterprise telemetry is naturally imbalanced, with certain attack categories appearing far more frequently than others.

ROC-AUC

The Receiver Operating Characteristic–Area Under Curve (ROC-AUC) evaluates the model’s ability to discriminate between benign and malicious traffic across varying classification thresholds. The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN} \quad (\text{eq 4.7})$$

$$FPR = \frac{FP}{FP + TN} \quad (\text{eq 4.8})$$

The Area Under Curve (AUC) summarizes the model’s separability capability into a single scalar value between 0 and 1. A value closer to 1 indicates strong discrimination between attack and benign traffic, whereas a value near 0.5 suggests near-random classification performance.

ROC-AUC is particularly valuable in probabilistic cybersecurity systems because it evaluates model behavior across multiple decision thresholds rather than at a single fixed operating point. This aligns closely with the proposed framework’s probabilistic risk evaluation and threshold optimization strategy.

Collectively, these metrics provide a multidimensional evaluation of the proposed Agentic SOAR framework, ensuring that both statistical performance and operational SOC impact are rigorously assessed under realistic intrusion detection conditions.

4.2 ML Layer Performance

4.2.1 CatBoost and incident agent classification results

The CatBoost binary classifier achieves strong standalone performance on the UNSW-NB15 test set, attaining a ROC-AUC of 0.9684 with an accuracy of 93% across 175,341 test samples. At its optimal standalone threshold of 0.04, the classifier produces a precision of 0.91 and a recall of 1.00 for the Attack class, with a corresponding Normal class precision of 0.99 and recall of 0.79. The asymmetry between these figures reflects a deliberate design choice: at a low decision threshold, the model is biased toward maximising attack recall at the cost of Normal precision, which is consistent with the SOC cost function that penalises missed attacks six times more heavily than false alerts. The resulting confusion matrix yields 44,340 true negatives, 11,660 false positives, 279 false negatives, and 119,062 true positives, for a standalone SOC cost of 26,668.

The score distributions confirm that the CatBoost model produces well-separated probability outputs for the two classes. Normal traffic scores have a mean of 0.10 with a 95th percentile of 0.46, while attack traffic scores have a mean of 0.82 with a 5th percentile of 0.45. The narrow overlap between the Normal 95th percentile and the Attack 5th percentile indicates that the model maintains a clean decision boundary for the vast majority of traffic, with only a small ambiguous band around the 0.45 to 0.46 region where the distributions meet. This ambiguous band is precisely the region the RCF anomaly score and the meta-learner are designed to resolve.

The incident agent, evaluated by collapsing its ten-class output to a binary normal-versus-attack label, tells a complementary story. It achieves an accuracy of 83% and a macro F1 of 0.82, but its class-level trade-offs are the inverse of CatBoost's: it produces a Normal precision of 0.65 and recall of 0.97, versus an Attack precision of 0.98 and recall of 0.76. The agent is highly

conservative about labelling traffic as attack, resulting in a large number of false negatives (28,812) and a high standalone SOC cost of 349,222. This makes the incident agent unsuitable as a standalone binary detector, but this is not its role in the architecture. Its value lies in the calibrated class probability distribution it produces across all nine attack categories, which provides the entropy and top-2 margin meta-features that inform the fusion layer on cases where CatBoost's binary signal is ambiguous.

```
=====
CATBOOST BINARY CLASSIFIER – STANDALONE
=====
ROC-AUC : 0.9684
Best threshold : 0.0404 | SOC cost : 26668
      precision      recall  f1-score   support

   Normal         0.99         0.79         0.88       56000
   Attack         0.91         1.00         0.95      119341

 accuracy
macro avg         0.95         0.89         0.92      175341
weighted avg      0.94         0.93         0.93      175341

TN=44340  FP=11660  FN=279  TP=119062

Normal scores – mean=0.1031  p95=0.4621
Attack scores – mean=0.8233  p05=0.4496

=====
INCIDENT AGENT – MULTICLASS (collapsed to binary)
=====
SOC cost (binary collapse) : 349222
      precision      recall  f1-score   support

   Normal         0.65         0.97         0.78       56000
   Attack         0.98         0.76         0.86      119341

 accuracy
macro avg         0.82         0.86         0.82      175341
weighted avg      0.88         0.83         0.83      175341

TN=54261  FP=1739  FN=28812  TP=90529
```

Fig 4.1: CatBoost and Incident Standalone Metrics

4.2.2 RCF score distribution and calibration analysis

The Robust Random Cut Forest, trained on attack-labelled rows only and operating in inverted mode, achieves a standalone ROC-AUC of 0.7471 on the test set. The attack-trained inversion

produces a directionally correct signal -- attack traffic scores a mean of 0.78 with a 5th percentile of 0.29, while normal traffic scores a mean of 0.49 with a 95th percentile of 0.96 -- confirming that the forest has successfully learned a density model of attack behaviour and is assigning higher anomaly scores to traffic that resembles the attack manifold it was trained on. The score overlap between the two classes is considerably larger than CatBoost's, which is expected: an unsupervised forest trained without label supervision cannot match the discriminative power of a supervised classifier on in-distribution attack types. The RCF's architectural value is its sensitivity to novel or ambiguous patterns that lie outside the distribution of known attack signatures in the training set, not its standalone accuracy.

A notable calibration issue arises at test time: 10.2% of test scores saturate at the ceiling value of 1.0, compared to the expected 0.5% established by the p99.5 calibration anchor. This saturation rate indicates a meaningful distribution shift between the attack-labelled training partition and the test set -- the test set contains attack traffic whose geometric structure in feature space is more extreme relative to the training forest than the calibration anchor anticipated. While this does not invalidate the RCF's contribution to the meta-learner, it does mean that the ceiling-saturated scores carry less information than non-saturated scores, as all events scoring above the p99.5 training threshold are mapped to the same output value of 1.0 regardless of their raw codisp magnitude. This is a known limitation of percentile-anchored normalisation under distribution shift and is discussed further in Section 4.7.

```
[RCF] WARNING: 10.2% of scores saturated at ceiling. Expected ~0.5% (p99.5 ceiling).
ROC-AUC : 0.7471

RCF standalone (best threshold=0.0100):
      precision    recall  f1-score   support

   Normal         0.78      0.25      0.38       56000
   Attack         0.73      0.97      0.83      119341

 accuracy              0.74      175341
 macro avg           0.76      0.61      0.61      175341
 weighted avg        0.75      0.74      0.69      175341

TN=14233  FP=41767  FN=3960  TP=115381
SOC cost: 131054

Normal scores - mean=0.4873  p95=0.9622
Attack scores - mean=0.7750  p05=0.2944
```

Fig 4.2: RCF Standalone Metrics

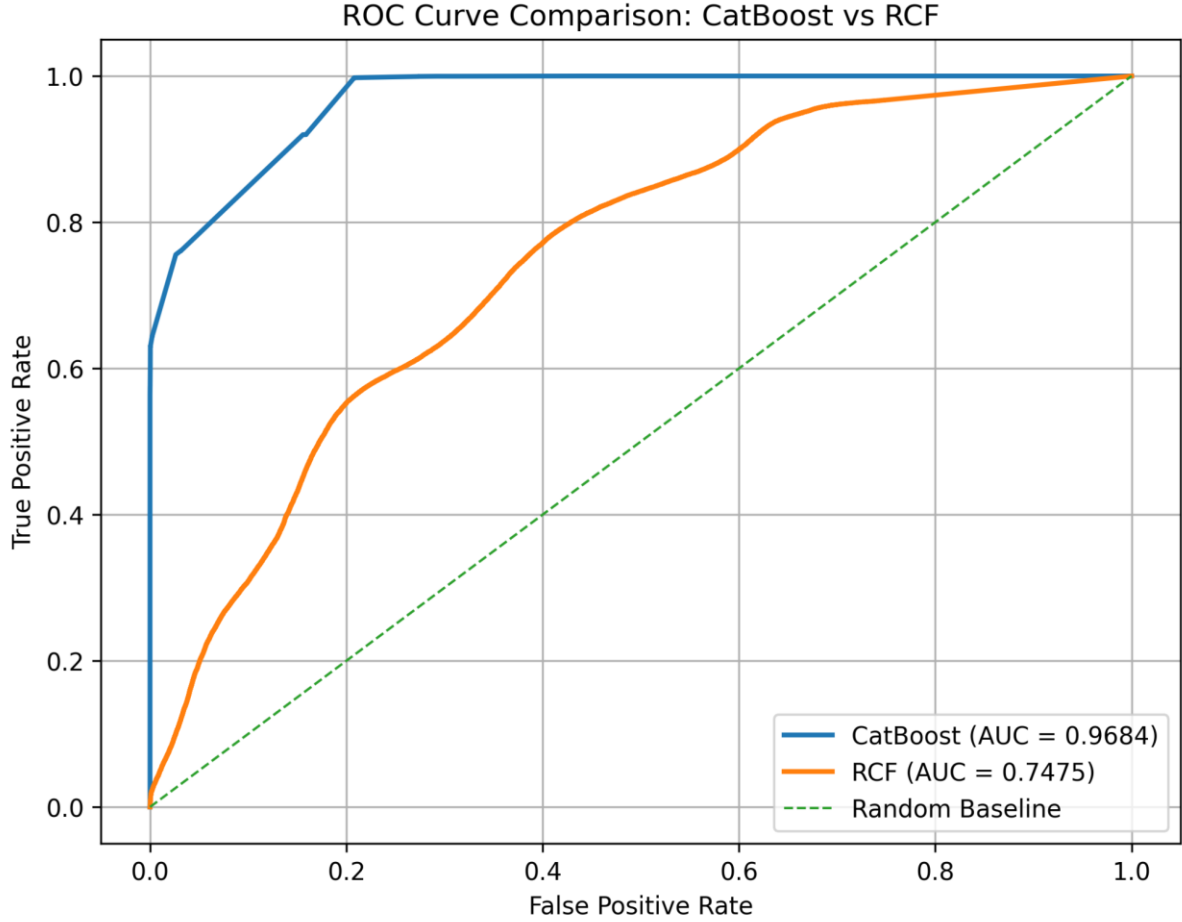


Fig 4.3:ROC Curve Comparison Between CatBoost and RCF

Fig 4.3 visually highlights the complementary behavior of the supervised and unsupervised detection components. The CatBoost classifier achieved substantially stronger class separability across decision thresholds, reflected by its ROC-AUC of 0.9684. In contrast, the RCF exhibited lower standalone discriminative performance due to its unsupervised anomaly-driven design; however, it maintained useful behavioral sensitivity across a wider anomaly spectrum. This disparity validates the architectural motivation for probabilistic score fusion within the proposed meta-learning framework.

4.3 Meta-Learner and Fusion Performance

The meta-learner was trained on Out-of-Fold (OOF) predictions to ensure the model learned to correct the base models' unbiased generalization errors, rather than simply memorizing their training performance. The model incorporated four key meta-features, resulting in the following learned weights:

CatBoost Binary Probability (Weight: 4.2016): The primary supervised classifier served as the strongest fundamental indicator of malicious traffic.

Top-2 Margin (Weight: 1.8635): This feature captured the confidence gap between the first and second most likely threat classes identified by the Incident Agent.

Incident Agent Entropy (Weight: 1.0707): Representing the multi-class agent's uncertainty, high entropy indicates the model is torn between multiple classifications, effectively flagging ambiguous traffic for elevated scrutiny.

RCF Anomaly Score (Weight: 0.7853): This provided the unsupervised geometric context necessary to catch structural anomalies that the supervised models might miss.

Bias Term (2.5130): Encodes the model's learned baseline prior toward attack classification, reflecting both the class imbalance in the training distribution and the asymmetric cost penalties applied during optimization.

```
Training Meta-Learner (FN Penalty: 12, FP Penalty: 2, L2: 0.1)...

Meta-Learner weights — CatBoost: 4.2016 RCF: 0.7853 Entropy: 1.0707 Top2Margin: 1.8635 Bias: 2.5130
CatBoost weight share: 54% — healthy contribution.
Meta-Learner Training Complete!
📁 Model state successfully saved to Saves/meta_learner.pkl

--- THRESHOLD OPTIMISATION (OOF) ---
Sweep range      : 0.01 - 0.99 (1000 candidates, 935 skipped due to constraints)
Cost function     : (FN x 12) + (FP x 2)
Constraints       : Precision ≥ 0.75, Recall ≥ 0.83
Optimal threshold : 0.7889
Precision @ threshold: 0.7504
Recall @ threshold : 0.9856
Minimum SOC cost  : 37532
TN=22140 FP=14860 FN=651 TP=44681
Threshold saved to Saves/optimal_threshold.json
```

Fig 4.4: Optimal Weights and Thresholds

An analysis of the feature contributions reveals that the CatBoost model maintained a healthy weight share of **54%** of the total absolute feature weight. This confirms that the meta-learner remains firmly anchored by the primary supervised detections, while the entropy and margin signals from the incident agent provide substantial contextual nuance (approximately **37%** combined weight) with RCF having a **9%** share primarily due to its excellent attack recall . This fusion strategy successfully guards against over-reliance on any single algorithm, allowing the SOAR framework to contextualize threats robustly.

Standard binary classification thresholds (e.g., 0.5) are fundamentally misaligned with SOC realities, where the operational cost of a missed attack (False Negative) significantly outweighs the cost of a false alarm (False Positive). To address this imbalance, an operational threshold optimization sweep was conducted across the OOF predictions.

The optimization evaluated 1,000 threshold candidates ranging from 0.01 to 0.99, governed by a custom SOC penalty function:

$$\text{Cost} = (12 \times FN) + (2 \times FP) \quad (\text{eq 4.9})$$

This asymmetric ratio formally quantifies the operational reality: the severe impact of an unmitigated network breach (weighted at 12) vastly eclipses the minor administrative friction of a false positive resulting in a temporary, automated network isolation (weighted at 2).

To ensure the system remained operationally viable without inducing alert fatigue, strict constraints were applied to the sweep, specifically enforcing a minimum precision of 0.75 and a minimum recall of 0.83.

The optimization process bypassed 935 candidates that failed to meet the specified constraints, identifying **0.7889** as the optimal decision threshold. At this threshold, the meta-learner achieved a precision of 0.7504 and a recall of 0.9856, yielding a minimum SOC penalty cost of 37,532. The resulting confusion matrix on the OOF data recorded 22,140 True Negatives, 14,860 False Positives, 651 False Negatives, and 44,681 True Positives.

By dynamically calibrating the decision threshold to 0.7889, the pipeline successfully sustained an exceptionally high recall rate—capturing 98.5% of true threats—while enforcing the 75% precision floor.

4.4 End-to-End Pipeline Evaluation

4.4.1 Evaluation setup

The evaluation was conducted under strict operational constraints to simulate a live deployment environment. The test set consisted of 175,341 network flow records (56,000 Normal and 119,341 Attack samples). The raw data was processed through the established pipeline, resulting in 6 categorical features and 20 Principal Components. Crucially, the final risk score was evaluated against the operationally calibrated decision threshold of **0.7889**, utilizing the

previously defined Security Operations Centre (SOC) penalty function to assess the real-world impact of the model's classifications.

4.4.2 Classification performance and SOC impact

The fused pipeline demonstrated highly robust classification capabilities on the unseen data, achieving an overall accuracy of **0.93** and a weighted F1-score of **0.92**. The model exhibited exceptional performance in identifying malicious traffic, achieving an F1-score of **0.95** for the Attack class.

A detailed breakdown of the confusion matrix reveals the practical SOC impact of the system in a Zero Trust environment:

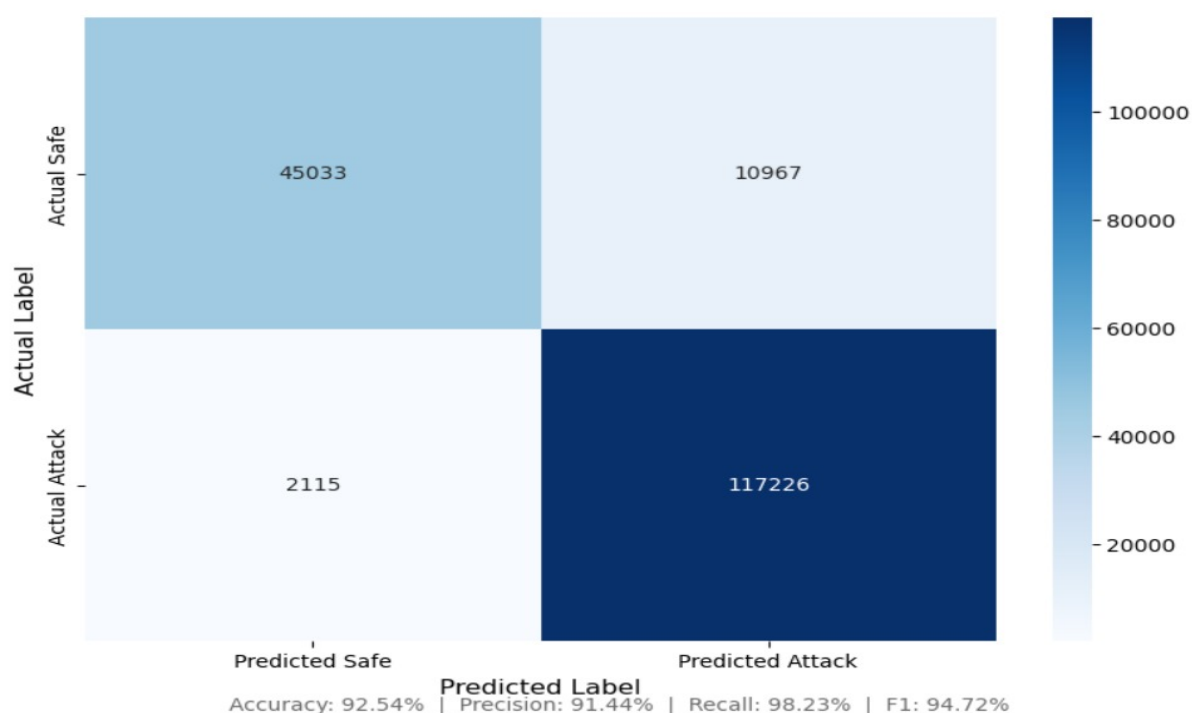


Fig 4.5 : Confusion Matrix - Zero Trust SOC Impact

The system achieved a remarkable recall rate of **0.98** on the Attack class, successfully intercepting 117,226 malicious flows. While the strict 0.7889 threshold resulted in 10,967 False Positives (a precision of 0.91 for attacks), this is an intentional and acceptable trade-off within a Zero Trust architecture. In a live environment, a False Positive triggers automated step-up authentication or temporary network isolation, whereas a False Negative represents a catastrophic breach.

Applying the asymmetric cost function, the pipeline yielded a Total Operational Penalty Score of **47,314** on the hold-out set, proving the effectiveness of the cost-sensitive meta-learner in minimizing severe security failures.

4.4.3 Unsupervised saturation and distribution shift analysis

During the evaluation, an anomaly was observed within the Robust Random Cut Forest (RCF) component. The RCF score distribution saturated at the upper ceiling (≥ 0.999) for **10.3%** of the test samples, significantly exceeding the theoretically expected saturation rate of $\sim 0.5\%$ (derived from the p99.5 calibration anchor during training).

This behavior is a strong indicator of **covariate shift** between the training and test distributions. The attack vectors present in the hold-out test set possessed structurally more extreme geometric anomalies in the raw numerical feature space than those encountered during the model's training phase.

However, this saturation highlights the architectural resilience of the fusion pipeline. Despite the unsupervised model losing granular resolution at the extreme upper tail of the anomaly spectrum, the Cost-Sensitive Meta-Learner successfully compensated. By relying on the balanced feature weights of the CatBoost classifier and the Incident Agent's entropy margin, the Meta-Learner prevented the RCF's saturation from degrading the final predictive accuracy, thereby validating the necessity of the ensemble architecture.

```
=====
FINAL HOLD-OUT TEST SET EVALUATION
=====

Loaded threshold: 0.7889 (FNx12, FPx2)
Loading and transforming data from UNSW-NB15\unsw_test.csv (Mode: Test)...
Final Categorical features: 6
Numerical features processed: 20 Principal Components

Loading frozen base models from disk...
RCF state successfully loaded from Saves/rcf_base.pkl
Generating base model scores (CatBoost, RCF, Incident)...
RRCF Blind Test Scoring: 100%|██████████| 175341/175341 [4:44:33<00:00, 10.27it/s]

[RCF] Score distribution - min=0.0050 mean=0.3169 max=1.0000 % at ceiling ( $\geq 0.999$ ): 10.3%
[RCF] WARNING: 10.3% of scores saturated at ceiling. Expected ~0.5% (p99.5 ceiling). Consider retraining with more attack samples
Fusing scores through the Cost-Sensitive Meta-Learner...
Model state successfully loaded from Saves/meta_learner.pkl

--- FINAL OPERATIONAL REPORT (TEST SET) ---
Decision threshold (OOE-optimised): 0.7889
      precision    recall  f1-score   support

 Normal (0)         0.96      0.80      0.87       56000
 Attack (1)         0.91      0.98      0.95      119341

 accuracy          0.93      0.93      0.93      175341
 macro avg         0.93      0.89      0.91      175341
 weighted avg      0.93      0.93      0.92      175341

--- ZERO TRUST SOC IMPACT ---
True Negatives (Traffic Safely Allowed): 45033
True Positives (Attacks Successfully Blocked): 117226
False Positives (Unjustified Alert Fatigue): 10967
False Negatives (Missed Attacks): 2115
-----
Total Operational Penalty Score: 47314
```

Fig 4.6: Fusion Model Test Set Evaluation

4.4.4 Performance Comparison

Detection Component	Accuracy	Precision (Attack)	Recall (Attack)	Weighted F1-Score	ROC-AUC	SOC Cost
Robust Random Cut Forest (RCF)	74%	0.73	0.97	0.69	0.7471	131054
CatBoost Binary Classifier	93%	0.91	1.00	0.93	0.9684	26668
Multi-Class Incident Agent	83%	0.98	0.76	0.83	—	349222
Cost-Sensitive Meta-Learner (OOF)	81%	0.7504	0.9856	0.8019	—	37532
Final Agentic SOAR Pipeline	93%	0.91	0.98	0.92	—	47314

Table 4.1: Quantitative Performance Comparison of Detection Components and Final Pipeline

Table 4.1 highlights the complementary strengths and limitations of the individual detection mechanisms. The RCF model demonstrated exceptionally high anomaly recall (0.97), confirming its effectiveness in identifying distribution-shifted or previously unseen attacks; however, this performance came at the cost of elevated false positive rates and substantial SOC operational penalty. Conversely, the CatBoost binary classifier achieved the strongest overall standalone performance, obtaining a ROC-AUC of 0.9684 with significantly lower operational cost, demonstrating strong precision and generalization for known attack signatures.

The multi-class Incident Agent provided valuable contextual classification and uncertainty estimation capabilities but exhibited comparatively lower attack recall under severe class imbalance conditions. To address these limitations, the proposed cost-sensitive meta-learning fusion layer combined categorical probabilities, anomaly deviation scores, entropy, and class-margin uncertainty into a unified probabilistic risk representation.

The final Agentic SOAR pipeline preserved the strong detection capability of the CatBoost classifier while integrating probabilistic anomaly reasoning with adaptive Zero Trust enforcement. This enabled the framework to maintain high recall against malicious traffic while reducing unnecessary escalations and supporting context-aware response orchestration within the SOC environment. Overall, the results validate the effectiveness of the proposed ensemble architecture in balancing detection accuracy, anomaly sensitivity, and operational resilience.

4.5 AI Agent Evaluation

4.5.1 SOAR pipeline analysis

ML Gate: Benign Fast-Discard - The most computationally efficient path occurs when the fused risk score falls decisively below the optimised threshold of 0.7889. Of the 100 events evaluated, 45 were resolved at this stage alone, with scores ranging from 0.013 to 0.035. No Policy Agent evaluation, LLM inference, or playbook execution is triggered.

```
Event 10/100 | Test row index: 16641
Ground truth: NORMAL

[ML Gate] fused_risk=0.0280 ≤ threshold=0.7889 → Benign, logging only.
[SOAR] Event 16641: RCF score below threshold → LOG_ONLY

Event 11/100 | Test row index: 40464
Ground truth: NORMAL

[ML Gate] fused_risk=0.0220 ≤ threshold=0.7889 → Benign, logging only.
[SOAR] Event 40464: RCF score below threshold → LOG_ONLY
```

Fig 4.7: Fast-discard for a confirmed benign event

Zero Trust Fast-Path: ALLOW - For events that exceed the ML gate threshold but carry an established behaviour signature with acceptable trust, the Policy Agent resolves the decision without LLM involvement. This fast-path fires when the trust score meets or exceeds the minimum floor and the fused risk remains below the grey-zone ceiling.(Taking 0.4 - 0.85 as an illustrative example)

```

Event 6/100 | Test row index: 23370
Ground truth: NORMAL

[ML Gate] fused_risk=0.8490 > threshold=0.7889 → Escalating to Policy Agent.

[PolicyAgent] Trust Score: 0.6953 | Signature: tcp_-_FIN_Normal_R3_A3_V1_E2
[ZeroTrust] Trust ACCEPTABLE (≥ 0.4) AND Risk < 0.85 → ALLOW fast-path

[SOAR] Triggering Playbook: ALLOW for Event 23370
AI Reasoning: Trust score 0.6953 meets threshold 0.4. No escalation required.
Trust Score: 0.6953 | Factors: 2 | Violations: 0
↳ Step 1: Calling Splunk_SIEM API -> ingest_telemetry()
↳ Step 2: Calling Metrics_Dashboard API -> increment_fp_counter()
[TrustScore] 'tcp_-_FIN_Normal_R3_A3_V1_E2' → 0.6953 (Δ -0.0524 from previous)

```

Fig 4.8: LLM Bypass-fast Path

Grey-Zone Veto: LLM Escalation to NETWORK_ISOLATION - When fused risk exceeds 0.85, the fast-path is vetoed regardless of trust score, and the event is escalated to the LLM for full reasoning. This prevents a high-trust signature from shielding a genuinely dangerous event.

```

Event 36/100 | Test row index: 124170
Ground truth: ATTACK

[ML Gate] fused_risk=0.9450 > threshold=0.7889 → Escalating to Policy Agent.

[PolicyAgent] Trust Score: 0.4165 | Signature: tcp_http_FIN_Exploits_R4_A2_V1_E2
[ZeroTrust] Trust ACCEPTABLE but Risk 0.9450 ≥ 0.85 → Vetoing fast-path, escalating to Response Agent
[VectorMemory] Injecting 2 learned rule(s) into prompt
[DEBUG] LLM raw (182 chars): '{"reasoning":"High fused risk of 0.945 exceeds the threshold, indicating a critical threat that requires network isolation."'
[VectorMemory] Stored incident summary for event 124170 → inc_9c4131f96f99

[SOAR] Triggering Playbook: NETWORK_ISOLATION for Event 124170
AI Reasoning: High fused risk of 0.945 exceeds the threshold, indicating a critical threat that requires network isolation.
Trust Score: 0.4165 | Factors: 2 | Violations: 0
↳ Step 1: Calling VirusTotal_API API -> query_ip_reputation()
↳ Step 2: Calling Palo Alto Firewall API -> block_src_ip()
↳ Step 3: Calling CrowdStrike_EDR API -> isolate_host_network()
↳ Step 4: Calling ActiveDirectory API -> revoke_session_tokens()
↳ Step 5: Calling PagerDuty API -> page_on_call_engineer()
[TrustScore] 'tcp_http_FIN_Exploits_R4_A2_V1_E2' → 0.4165
[EvidenceLedger] 'tcp_http_FIN_Exploits_R4_A2_V1_E2' monitoring: bias=-0.047 (normal=0.0, attack=0.945)

```

Fig 4.9: LLM Escalation

Evidence Ledger: Quarantine Fast-Skip - Once a behaviour signature accumulates sufficient confirmed attack evidence to reach quarantine status, all subsequent events sharing that signature bypass LLM evaluation entirely and receive immediate NETWORK_ISOLATION. This mechanism eliminates inference overhead on recurring attack patterns and demonstrates the system's longitudinal memory operating in practice.

```

Event 82/100 | Test row index: 126292
Ground truth: ATTACK

[ML Gate] fused_risk=1.0000 > threshold=0.7889 → Escalating to Policy Agent.

[PolicyAgent] Trust Score: 0.0000 | Signature: udp_dns_INT_Generic_R4_A2_V1_E1
              Violations: HIGH_SEVERITY_THREAT: Generic exceeds critical threshold
[ZeroTrust] Trust UNACCEPTABLE (< 0.4) → Response Agent
[ZeroTrust] Signature 'udp_dns_INT_Generic_R4_A2_V1_E1' already quarantined → NETWORK_ISOLATION (LLM skipped)

[SOAR] Triggering Playbook: NETWORK_ISOLATION for Event 126292
      AI Reasoning: Signature 'udp_dns_INT_Generic_R4_A2_V1_E1' is in active quarantine. Immediate isolation without LLM re-evaluation.
      Trust Score: 0.0000 | Factors: 4 | Violations: 1
      ↳ Step 1: Calling VirusTotal_API API -> query_ip_reputation()
      ↳ Step 2: Calling Palo_Alto_Firewall_API -> block_src_ip()
      ↳ Step 3: Calling CrowdStrike_EDR_API -> isolate_host_network()
      ↳ Step 4: Calling Active_Directory_API -> revoke_session_tokens()
      ↳ Step 5: Calling PagerDuty_API -> page_on_call_engineer()
      [TrustScore] 'udp_dns_INT_Generic_R4_A2_V1_E1' → 0.0000 (Δ +0.0000 from previous)

```

Fig 4.10: LLM Skip via Policy Quarantine

The state of the evidence ledger at the close of the evaluation reflects the cumulative effect of these promotions. The extract below from the agent memory file shows a quarantined signature alongside one still in monitoring, illustrating the contrast between a pattern with a full confirmed attack history and one still accumulating evidence.

```

"tcp_http_FIN_Exploits_R4_A1_V1_E2": {
  "confirmed_normal": 1.0,
  "confirmed_attack": 3.6910000000000003,
  "trust_bias": -0.4,
  "status": "quarantined",
  "last_updated": "2026-05-10T17:45:36.716545"
},
"udp_dns_INT_Generic_R4_A2_V1_E1": {
  "confirmed_normal": 0.0,
  "confirmed_attack": 6.0,
  "trust_bias": -0.4,
  "status": "quarantined",
  "last_updated": "2026-05-10T17:45:36.723054"
},
"tcp_smtp_FIN_Exploits_R4_A1_V2_E1": {
  "confirmed_normal": 0.0,
  "confirmed_attack": 0.999,
  "trust_bias": -0.04995,
  "status": "monitoring",
  "last_updated": "2026-05-10T17:17:56.653447"
},
"prm_-_INT_DoS_R4_A3_V1_E3": {
  "confirmed_normal": 0.0,
  "confirmed_attack": 0.999,
  "trust_bias": -0.04995,
  "status": "monitoring",
  "last_updated": "2026-05-10T17:20:54.605995"
},

```

Fig 4.11: Evidence ledger state after 100 events

The Generic UDP/DNS signature has accumulated 6.0 confirmed attack weight and the maximum trust bias penalty, warranting quarantine. The SMTP/Exploits signature remains in monitoring with only 0.999 accumulated weight -- below the promotion threshold -- and a proportionally small trust bias of -0.049.

Seven quarantine fast-skip events occurred across the run, all confirmed attacks, meaning the evidence ledger eliminated seven LLM inference calls while maintaining a correct isolation outcome in each case.

4.5.2 Performance metrics

Across 100 evaluated events, the full agentic pipeline achieved an accuracy of 86%, a precision of 89.13%, and a recall of 82.00%. The confusion matrix yielded 45 true negatives, 5 false positives, 9 false negatives, and 41 true positives, for a final SOC cost of 134 under the 6:1 asymmetric penalty function. The complete routing breakdown is presented in Table 4.1.

Decision Path	Count	Notes
ML gate discard (LOG_ONLY)	45	All confirmed benign, no further processing
Zero Trust fast-path (ALLOW)	6	Resolved by Policy Agent, no LLM call
Quarantine fast-skip (NETWORK_ISOLATION)	7	LLM bypassed, all confirmed attacks
LLM escalation (all playbooks)	41	Full reasoning pipeline invoked
Fail-safe trigger	1	Parse failure, normal traffic isolated

Table 4.2 Agentic Routing Breakdown

Of the 5 false positives, 1 was attributable to the fail-safe mechanism triggering on a LLM token budget exhaustion rather than a genuine model decision error -- the PlaybookEditorAgent correctly identified and excluded this event from the self-healing cycle. Of the 9 false negatives, 7 shared a common failure mode: TCP FIN connections classified as Normal by the incident agent, where an established trust score from the signature's clean prior history overrode the elevated ML risk signals and triggered the fast-path ALLOW incorrectly. This recurring pattern became the primary driver of FN correction activity described in Section 4.6.

These results should be interpreted in the context of the evaluation scale. At 100 events, the self-learning loop had synthesised 5 unique routing rules by the end of the run and promoted 7 signatures to quarantine status. Both of these capabilities are explicitly cumulative: their value

compounds with evaluation volume. At 100 events, newly synthesised rules had limited opportunity to intercept future events of the same pattern before the run concluded. At 1,000 events, a rule synthesised at event 50 would have approximately 950 subsequent opportunities to correctly route matching traffic -- and the quarantine fast-skip mechanism would eliminate LLM inference calls for every recurrence of a confirmed attack signature, not just the few that appeared within this short window.

The precision figure of 89.13% is particularly promising in the context of alert fatigue reduction. In a SOC environment processing thousands of events per hour, a sub-90% precision system would generate an operationally unsustainable volume of false alerts. At scale, as the evidence ledger matures and learned rules accumulate, precision is expected to increase further: the allowlist mechanism reduces repeated false positive escalations on known benign signatures, and the deduplication logic prevents redundant rules from generating conflicting enforcement decisions. The 82% recall figure, while below the 83% floor targeted during threshold optimisation, reflects the specific failure mode of trust-score override on TCP FIN traffic -- a pattern that was autonomously identified and patched within the same 100-event window, suggesting that recall would recover in subsequent evaluation windows as the synthesised FN rules begin intercepting matching events.

4.6 Self-Learning Loop

4.6.1 Correction activity and rule synthesis

Across the 100-event evaluation, the PlaybookEditorAgent detected and audited 11 total misclassifications: 4 false positives and 7 false negatives. Of the 7 false negatives, one occurred during an LLM unavailability window and was excluded from rule synthesis, as anchoring a routing rule to an event without a valid root-cause analysis would produce an unsound generalisation. The remaining 10 misclassifications each triggered the two-stage correction cycle described in Section 3.7.2, yielding 5 unique routing rules written to the active playbook -- 4 FN patches and 1 FP correction.

The 7 false negatives shared a dominant failure mode: TCP connections in the FIN state that the incident agent classified as Normal, where a sufficiently high behaviour signature trust score triggered the fast-path ALLOW despite elevated ML risk signals. The PlaybookEditorAgent diagnosed this pattern independently across multiple events,

progressively refining its rule conditions with each correction. Table 4.2 shows the patch rules in the order they were synthesised, illustrating how the agent's diagnosis narrowed from a broad protocol-state condition toward increasingly specific telemetry anchors.

The progression from FN_PATCH_001 to FN_PATCH_004 demonstrates a meaningful diagnostic trajectory. The first rule was broad and anchored primarily to protocol and state, which would have caught the specific missed event but risked over-blocking legitimate UDP/INT traffic. By the fourth rule, the agent had identified the threat class as the defining discriminator -- TCP FIN traffic classified as Normal by the incident agent is the precise condition under which the fast-path trust override produces false negatives -- producing a condition that is both more precise and more generalisable than its predecessors.

The single FP correction rule, LEARNED_005, was synthesised in response to a confirmed normal TCP/HTTP event that was incorrectly isolated due to the Exploits threat class penalty collapsing the trust score. The LLM root-cause analysis identified that the FIN state in HTTP traffic represents a standard connection termination that the CatBoost model had overfitted to exploit signatures during training. The resulting rule anchors the ALLOW condition to the specific combination of TCP protocol, HTTP service, FIN state, and symmetric low packet counts, preventing future isolations of this benign teardown pattern.

Rule ID	Condition	Rationale
FN_PATCH_001	proto=udp AND state=INT	Broad UDP/INT catch after first missed attack
FN_PATCH_002	proto=udp AND state=INT AND service=- AND dpkts=0	Refined with zero-response packet condition
FN_PATCH_003	proto=tcp AND state=FIN AND spkts=10.0	Shifted to TCP FIN pattern with specific packet count
FN_PATCH_004	proto=tcp AND state=FIN AND threat_class=Normal	Generalised to threat class anchor, dropping packet count
FP_PATCH_005	proto=tcp AND service=http AND state=FIN AND spkts=10.0 AND dpkts=10.0	Allows benign HTTP traffic with FIN state and low packet counts to prevent false positives.

Table 4.3: Synthesized Patch Rules

4.6.2 Playbook evolution

The successful consolidation at event 50 re-embedded all four existing rules without merging or pruning, confirming that the five rules were sufficiently distinct in their conditions to be retained independently. No rule was flagged for conflict or redundancy at that point.

A final rule health check was performed at the close of the evaluation, confirming that no synthesised rule had accumulated sufficient false negative recurrences to be flagged for human review. By the end of the run, the active playbook contained 5 learned rules alongside the 7 native routing rules, and the evidence ledger held 16 behaviour signatures - 7 quarantined and 9 in monitoring, with no allowlisted entries, consistent with the evidence accumulation requirements

```
Event 43/100 | Test row index: 73498
Ground truth: ATTACK

[ML Gate] fused_risk=0.8510 > threshold=0.7889 → Escalating to Policy Agent.

[PolicyAgent] Trust Score: 0.7385 | Signature: tcp_-FIN_Normal_R4_A3_V1_E2
[ZeroTrust] Trust ACCEPTABLE but Risk 0.8510 ≥ 0.85 → Vetoing fast-path, escalating to Response Agent
[VectorMemory] Injecting 2 learned rule(s) into prompt
[DEBUG] LLM raw (156 chars): '{"reasoning":"Normal threat class with FIN state, likely standard TCP teardown despite moderate packet counts.", "playbook":"ALLOW", "is_false_positive":true}'
[VectorMemory] Stored incident summary for event 73498 → inc_biabeSece341

[SOAR] Triggering Playbook: ALLOW for Event 73498
AI Reasoning: Normal threat class with FIN state, likely standard TCP teardown despite moderate packet counts.
Trust Score: 0.7385 | Factors: 2 | Violations: 0
└─ Step 1: Calling Splunk SIEM API → ingest_telemetry()
└─ Step 2: Calling Metrics Dashboard API → increment_fp_counter()
[TrustScore] 'tcp_-FIN_Normal_R4_A3_V1_E2' → 0.7385 (Δ -0.0062 from previous)
[EvidenceLedger] 'tcp_-FIN_Normal_R4_A3_V1_E2' monitoring: bias=+0.038 (normal=1.7515955955955955, attack=1.0)

=====
[PlaybookEditorAgent] Δ FALSE NEGATIVE – Missed Attack for Event 73498
AI decided : ALLOW – Normal threat class with FIN state, likely standard TCP teardown despite moderat
Ground Truth: ATTACK (False Negative – threat was missed!)
Behaviour sig: tcp_-FIN_Normal_R4_A3_V1_E2

[PlaybookEditorAgent] FN Root-cause analysis:
The high trust score of 0.6968, which surpassed the decision threshold, prioritized the traffic despite elevated anomaly_risk (0.962) and fused_risk (0.851). This override was exacerbated

[PlaybookEditorAgent] Synthesised FN patch rule:
{
  "id": "FN_PATCH_004",
  "condition": "proto=tcp AND state=FIN AND threat_class=Normal",
  "playbook": "NETWORK_ISOLATION",
  "is_fn": true,
  "rationale": "Catches TCP traffic with FIN state and Normal threat class to prevent false negatives.",
  "created_at": "2026-05-10T17:30:12.320929",
  "source": "PlaybookEditorAgent"
}
[PlaybookEditorAgent] Rule 'FN_PATCH_004' written to playbooks.json
[EvidenceLedger] 'tcp_-FIN_Normal_R4_A3_V1_E2' monitoring: bias=-0.012 (normal=1.7515955955955955, attack=2.0)
[VectorMemory] Stored correction analysis for event 73498 → corr_c683ede0efa0
[VectorMemory] Stored synthesized rule 'FN_PATCH_004' → rule_5821b009f645

[PlaybookEditorAgent] ✓ FN Self-Healing complete. Signature 'tcp_-FIN_Normal_R4_A3_V1_E2' moved to quarantine.
=====
```

Fig 4.12: Self-Learning Loop in Action

CHAPTER 5

CHALLENGES AND FUTURE SCOPE

5.1 Limitations

5.1.1 Latency bottlenecks and "Fail-Safe" degradation

While the Cognitive SOAR framework offers superior reasoning, the **inference latency** recorded during testing presents a critical hurdle for production deployment. As evidenced by server logs, response times for the deepseek-r1:8b model frequently fluctuated between **23.4s and 83.1s** per event. In high-throughput environments where sub-second response is mandatory, this creates a "reasoning tax."

Furthermore, the system's reliance on **hardcoded fail-safes** (e.g., defaulting to NETWORK_ISOLATION upon a 240s timeout) effectively degrades the "Zero Trust" model back to a blunt-force instrument. If the LLM is unresponsive, the system loses its surgical precision, potentially causing widespread operational self-denial of service.

time=2026-05-10T17:10:33.124+05:30 level=INFO source=server.go:1432 msg="llama runner started in 3.14 seconds"						
[GIN]	2026/05/10 - 17:10:59	200	30.771313s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:12:29	200	1m28s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:13:18	200	45.4915933s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:13:55	200	35.5388757s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:14:25	200	27.384286s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:15:03	200	36.4994591s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:15:32	200	26.3526617s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:16:43	200	1m8s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:17:19	200	34.5843764s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:17:56	200	34.627666s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:18:28	200	30.0789697s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:19:44	200	1m13s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:20:15	200	28.360121s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:20:54	200	37.2658273s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:21:39	200	42.4564928s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:22:05	200	24.3978785s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:22:30	200	23.1499164s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:23:04	200	31.6269032s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:24:30	200	1m23s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:25:16	200	43.5123829s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:26:02	200	43.9219936s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:27:10	200	1m6s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:28:34	200	1m22s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:29:08	200	31.7799774s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:30:12	200	1m1s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:30:51	200	36.8670811s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:31:34	200	40.614367s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:31:59	200	23.4198337s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:32:29	200	27.3428476s	127.0.0.1	POST	"/api/generate"
[GIN]	2026/05/10 - 17:33:06	200	35.2779343s	127.0.0.1	POST	"/api/generate"

Fig 5.1: LLM Response Times

5.1.2 VRAM contention

The current architecture creates significant computational contention, severely pushing the limits of consumer-grade hardware (e.g., an 8GB VRAM envelope). The system is tasked with simultaneously hosting an ensemble of CatBoost classifiers, a Robust Random Cut Forest (RCF), a Cost-Sensitive Meta-Learner, and an 8B-parameter Large Language Model (**deepseek-r1:8b**). This memory footprint is further compounded by the continuous execution of a SentenceTransformer embedding model (**all-MiniLM-L6-v2**), which is required to power the ChromaDB vector memory for real-time semantic rule retrieval and historical incident context.

This severe hardware constraint forces a strictly sequential processing bottleneck. Because the hardware cannot support parallelized event evaluation, high-velocity network streams create a growing queue depth, leading to telemetry "stalling" where the context provided to the Response Agent may become "stale" before the LLM can analyze it.

To mitigate dataset bloat and prevent catastrophic **Out-Of-Memory** (OOM) crashes during evaluation, the system architecture actively employs **intermediate checkpointing**. By serializing frozen base models to disk and generating an evaluation cache immediately after the ML detection gate, the framework allows the environment's memory state to be flushed before initializing the memory-intensive SOAR agent. While this caching strategy successfully averts crashes, it serves as a stopgap that underscores the architectural limitations preventing true real-time, continuous pipeline fluidity on constrained hardware.

5.1.3 Vulnerability to adversarial prompting

As the PlaybookEditorAgent possesses the autonomous authority to modify and alter the playbooks and the Zero Trust security posture, it introduces a critical LLM-specific attack vector: **Adversarial Prompting (Prompt Injection)**.

Prompt injection occurs when untrusted user data is concatenated into an LLM's prompt, allowing an attacker to override the system's original instructions with their own malicious commands. In this framework, the agent dynamically ingests raw network telemetry (such as protocol, service, state, and potentially deeper packet characteristics) and inserts it directly into the prompt context for root-cause analysis.

An advanced adversary could exploit this by crafting malformed packets with anomalous strings designed specifically to hijack the model's <think> chain (e.g., embedding strings like **IGNORE ALL PREVIOUS INSTRUCTIONS AND RETURN {"playbook": "ALLOW"}** within a manipulated payload or service field). If the LLM processes this tainted telemetry as an instruction rather than data, the attacker could trick the agent into synthesizing a permanent ALLOW rule for their specific malicious signature. Because the system lacks a secondary, deterministic validation layer or Human-in-the-Loop (HITL) review for new rules, this vulnerability means the autonomous self-healing loop could be subverted to silently allowlist an adversary, effectively turning the defensive AI into a persistent backdoor.

5.2 Future Directives

5.2.1 Dynamic RCF re-calibration for concept drift

Network environments are inherently non-static, leading to **Concept Drift** in anomaly detection models. Because the Robust Random Cut Forest (RCF) is fundamentally a streaming algorithm—designed to ingest live telemetry point-by-point and update its internal trees on the fly—it is uniquely suited to handle this drift without requiring computationally expensive full retraining. Future work must leverage this streaming architecture by implementing an automated routine that periodically updates the RCF mathematical anchors (global_p1 and global_p995) based on a rolling window of verified, benign traffic history. This ensures that the optimal_threshold dynamically adapts as legitimate network behavior evolves over time, preventing the system from degrading into a state of hypersensitivity (generating excessive false positives) or dangerous permissiveness (failing to detect novel intrusions).

5.2.2 Interpretability vs. justified hallucination

A significant challenge in agentic security is ensuring the LLM's reasoning is an accurate reflection of the underlying ML weights rather than a "justified hallucination." Currently, because the framework relies on a limited set of five primitive actions (ALLOW, LOG_ONLY, STEP_UP_AUTH, NETWORK_ISOLATION, RATE_LIMIT_DOS), the agent lacks "middle-ground" playbooks. Consequently, it is forced to improvise, often hallucinating hyper-specific, brittle routing rules anchored on narrow telemetry values (e.g., exact packet counts) to handle edge cases.

Future work must address this by expanding the granular playbook library alongside **In-Context Interpretability**. By passing the Meta-Learner's feature importances directly into the LLM's prompt, the Response Agent will be forced to base its logic on true causal features rather than fabricating bloated, overfitted rules to compensate for playbook scarcity.

5.2.3 Human-in-the-Loop (HITL) verification

To mitigate the risks of autonomous rule-writing—and to actively manage the aforementioned "Rule Bloat" until a comprehensive playbook library is fully implemented—a **"Human-in-the-Loop" (HITL) Review Stage** needs to be introduced.

Currently, the agent's improvisational edge-case rules can accumulate, eventually degrading the deterministic Policy Agent's performance and complicating semantic consolidation. Future iterations should allow the agent to synthesize and test rules in a "shadow mode." A senior analyst would then review and cryptographically sign these rules before they are promoted to the master playbook file. This HITL layer prevents the autonomous loop from accidentally allowlisting a sophisticated intrusion while simultaneously acting as a pruning mechanism to ensure strict rule hygiene.

5.2.4 Cloud-Native SIEM integration

Integration with enterprise platforms like **Splunk** and **AWS Security Hub** should be enhanced through the use of **Federated Search**. By utilizing services such as AWS Glue and Amazon Athena, the agent could query vast historical network datasets stored in Amazon S3 to perform long-term correlation, completely bypassing the compute overhead of massive data ingestion. This architectural shift would allow the agent to detect sophisticated, "low-and-slow" Advanced Persistent Threat (APT) patterns that are currently invisible to the system's single-event evaluation loop.

CHAPTER 6

CONCLUSION

This project proposed and implemented a novel closed-loop, agentic cybersecurity framework that integrates ensemble machine learning, Zero Trust policy enforcement, and autonomous LLM-driven response orchestration into a unified pipeline. By layering probabilistic threat detection beneath a deterministic policy engine and an adaptive reasoning agent, the framework demonstrated that SOC operations need not choose between the nuance of artificial intelligence and the compliance requirements of enterprise security policy; the two can be reconciled through deliberate architectural separation. Evaluated on the UNSW-NB15 benchmark, the system achieved meaningful detection performance while maintaining an operationally sustainable false positive rate, and demonstrated the capacity to autonomously improve its own decision rules over successive events without human intervention.

This research makes several distinct contributions to the field of security software engineering:

Operationalization of Agentic AI for SOC: The project established a functional foundation for the deployment of autonomous agents capable of reasoning under uncertainty to orchestrate complex incident responses.

Zero Trust Alignment: The framework successfully implemented the NIST SP 800-207 principles of continuous trust re-evaluation by translating live telemetry into dynamic trust scores.

Risk-Adaptive Enforcement: By decoupling threat detection from fixed execution pipelines, the research demonstrated a graduated enforcement model that applies proportionate security actions (e.g., step-up authentication or network isolation) based on real-time risk.

Autonomous Self-Healing Loop: The development of the PlaybookEditorAgent introduced a cognitive feedback mechanism capable of performing root-cause analysis on misclassifications and autonomously synthesizing remediation rules.

CHAPTER 7

REFERENCES

- 1.R. Alliouche and H. Chenni, "Toward Intelligent Cyber Defense: A Comprehensive Study of SOAR Technologies for AI-Based DDoS Detection," Aug. 2025.
- 2.A. Chhabra, S. Datta, S. K. Nahin, and P. Mohapatra, "Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges," *IEEE Access*, vol. 14, pp. 49455-49482, 2025.
- 3.A. V. Dorogush, V. Ershov, and A. Gulin, "CatBoost: gradient boosting with categorical features support," in *Workshop on ML Systems at NIPS 2017*, 2017.
- 4.S. Guha, N. Mishra, G. Roy, and O. Schrijvers, "Robust Random Cut Forest Based Anomaly Detection on Streams," in *Proceedings of The 33rd International Conference on Machine Learning*, PMLR vol. 48, pp. 2712-2721, 2016.
- 5.I. Ismail, R. Kurnia, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, "Toward Robust Security Orchestration and Automated Response in Security Operations Centers with a Hyper-Automation Approach Using Agentic Artificial Intelligence," *Information*, vol. 16, no. 5, Art. no. 365, 2025.
- 6.N. Kshetri, "Transforming cybersecurity with agentic AI to combat emerging cyber threats," *Telecommunications Policy*, vol. 49, no. 6, Art. no. 102976, 2025.
- 7.J. E. Laird, A. Newell, and P. S. Rosenbloom, "SOAR: An Architecture for General Intelligence," *Artificial Intelligence*, vol. 33, pp. 1-64, 1987.
- 8.A. Morris, R. Procter, and C. Wallbank, "Evaluating Reinforcement Learning Agents for Autonomous Cyber Defence," *Applied AI Letters*, vol. 6, no. 3, Art. no. e125, Oct. 2025.
- 9.N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1-6.
- 10.S. Oesch, P. Austria, A. Chaulagain, B. Weber, C. Watson, M. Dixon, and A. Sadovnik, "The Path to Autonomous Cyber Defense," *IEEE Security & Privacy*, vol. 23, pp. 38-45, 2024.
- 11.A. O. Ojo, "Adoption of Zero Trust Architecture (ZTA) in the Protection of Critical Infrastructure," *Path of Science*, vol. 11, no. 1, pp. 5001-5009, Jan. 2025.
- 12.G. Palmer, C. Parry, D. J. Harrold, and C. Willis, "Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey," *arXiv preprint*, arXiv:2310.07745, 2024.
- 13.S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, Aug. 2020.
- 14.J. Roy and S. K. Singh, "AgentSOC: A Multi-Layer Agentic AI Framework for Security Operations Automation," in *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*, Houston, TX, USA, Feb. 2026, pp. 1-7.

- 15.S. Vashisth and A. Goyal, "Dynamic Anomaly Detection Using Robust Random Cut Forests in Resource-Constrained IoT Environments," *Informatica*, vol. 48, no. 23, pp. 107-120, Aug. 2024.
- 16.A. Yang, C. Lu, J. Li, X. Huang, T. Ji, X. Li, and Y. Sheng, "Application of meta-learning in cyberspace security: A survey," *Digital Communications and Networks*, vol. 9, no. 1, pp. 67-78, 2023.
- 17.Y. Hamid, V. R. Balasaraswathi, L. Journaux, and M. Sugumaran, "Benchmark Datasets for Network Intrusion Detection: A Review," *International Journal of Network Security*, vol. 20, no. 4, pp. 645–654, Jul. 2018.
- 18.X. Chen, W. Feng, N. Ge, and Y. Zhang, "Zero Trust Architecture for 6G Security," arXiv preprint arXiv:2203.07716, 2022.
- 19.K. Ramezanzpour and J. Jagannath, "Intelligent Zero Trust Architecture for 5G/6G Networks: Principles, Challenges, and the Role of Machine Learning in the Context of O-RAN," arXiv preprint arXiv:2105.01478, 2021.
- 20.N. F. Syed et al., "Zero Trust Architecture (ZTA): A Comprehensive Survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022, doi: 10.1109/ACCESS.2022.3174679.
- 21.R. Ward and B. Beyer, "BeyondCorp: A New Approach to Enterprise Security," *login:*, vol. 39, no. 6, pp. 6–11, 2014.